

```
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head>
4   <meta charset="utf-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>Grifon - Grafo Interativo v3.6.0</title>
7   <link rel="icon" type="image/png" href="img/icon.png">
8   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel=
  "stylesheet">
9   <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
  ></script>
10  <link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.css"
  rel="stylesheet">
11  <script src="https://unpkg.com/cytoscape/dist/cytoscape.min.js"></script>
12  <style>
13    /* =====
14     VARIÁVEIS DE TEMA - Definem as cores principais do sistema
15     ===== */
16    :root {
17      --primaria: #0d6efd;
18      --primaria-escuro: #0b5ed7;
19      --fundo: #f0f2f5;
20      --fundo-card: #ffffff;
21      --borda: #dee2e6;
22      --texto: #1e293b;
23      --texto-secundario: #334155;
24      --sombra: 0 8px 24px rgba(0, 0, 0, 0.05);
25    }
26
27    * {
28      box-sizing: border-box;
29    }
30
31    html, body {
32      height: 100%;
33      overflow: hidden;
34      background: var(--fundo);
35      font-family: 'Segoe UI', system-ui, -apple-system, sans-serif;
36    }
37
38    /* Container principal do grafo */
39    #cy {
40      height: calc(95vh - 150px);
41      border: 2px solid var(--borda);
42      border-radius: 16px;
43      background: linear-gradient(180deg, #ffffff 0%, #f8faff 100%);
44      position: relative;
45      box-shadow: var(--sombra);
46    }
47
48    /* Marca d'água do Grifon no canto superior direito */
49    .grifon-bg {
50      position: absolute;
51      top: 11px;
52      right: 20px;
53      width: 110px;
54      height: 110px;
55      background-image: url('img/grifon.png');
56      background-repeat: no-repeat;
57      background-size: contain;
58      background-position: center;
59      z-index: 10;
60      pointer-events: none;
61    }
62
63    .small-label {
64      font-size: 0.78rem;
65      font-weight: 500;
66      color: var(--texto-secundario);
```

```

67     }
68
69     /* Botão flutuante para abrir o painel lateral */
70     .fab-button {
71         position: fixed;
72         top: 8px;
73         left: 16px;
74         z-index: 1085;
75         width: 48px;
76         height: 48px;
77         border-radius: 50%;
78         background: var(--primaria);
79         border: none;
80         color: white;
81         box-shadow: 0 8px 20px rgba(13, 110, 253, 0.3);
82         transition: all 0.2s;
83         display: flex;
84         align-items: center;
85         justify-content: center;
86         font-size: 1.4rem;
87         cursor: pointer;
88     }
89
90     .fab-button:hover {
91         background: var(--primaria-escura);
92         transform: scale(1.05);
93     }
94
95     /* Barra de ferramentas flutuante sobre o grafo */
96     .toolbar-stage {
97         position: absolute;
98         top: 20px;
99         left: 16px;
100        z-index: 1040;
101        display: flex;
102        gap: 8px;
103        flex-wrap: wrap;
104        background: rgba(255, 255, 255, 0.95);
105        backdrop-filter: blur(10px);
106        border: 1px solid #e9ecef;
107        border-radius: 40px;
108        padding: 6px 10px;
109        box-shadow: 0 12px 28px rgba(0, 0, 0, 0.08);
110    }
111
112    /* Destaca o modo ativo na barra de ferramentas */
113    .mode-active {
114        background: var(--primaria) !important;
115        color: white !important;
116        border-color: var(--primaria) !important;
117    }
118
119    /* Menu de contexto (clique direito) */
120    .ctx-menu {
121        position: fixed;
122        z-index: 2000;
123        min-width: 220px;
124        background: white;
125        border: 1px solid var(--borda);
126        border-radius: 16px;
127        box-shadow: 0 20px 35px rgba(0, 0, 0, 0.2);
128        display: none;
129        overflow: hidden;
130        backdrop-filter: blur(10px);
131    }
132
133    .ctx-menu button {
134        width: 100%;
135        border: 0;

```

```
136     background: transparent;
137     text-align: left;
138     padding: 10px 16px;
139     font-size: 0.9rem;
140     transition: background 0.1s;
141     cursor: pointer;
142 }
143
144 .ctx-menu button:hover {
145     background: #f1f5ff;
146 }
147
148 .ctx-menu button:disabled {
149     opacity: 0.5;
150     cursor: not-allowed;
151 }
152
153 .ctx-menu .divider {
154     height: 1px;
155     background: var(--borda);
156     margin: 4px 0;
157 }
158
159 .offcanvas {
160     background: #ffffff;
161 }
162
163 /* Card de informações estatísticas do grafo */
164 .info-card {
165     background: #f8f9fa;
166     border-radius: 16px;
167     padding: 0.75rem;
168     margin-bottom: 1rem;
169     border-left: 4px solid var(--primaria);
170 }
171
172 .info-card .small {
173     font-size: 0.7rem;
174     text-transform: uppercase;
175     letter-spacing: 0.5px;
176 }
177
178 .info-card .value {
179     font-size: 1.1rem;
180     font-weight: 600;
181 }
182
183 /* Painel de logs no canto inferior direito */
184 .log-panel {
185     position: absolute;
186     bottom: 60px;
187     right: 20px;
188     width: 420px;
189     max-height: 400px;
190     min-height: 300px;
191     background: #1e293b;
192     color: #a5f3fc;
193     font-family: 'JetBrains Mono', 'Courier New', monospace;
194     font-size: 0.8rem;
195     padding: 0;
196     border-radius: 16px;
197     border: 1px solid #334155;
198     box-shadow: 0 20px 35px rgba(0, 0, 0, 0.35);
199     z-index: 1050;
200     overflow: hidden;
201     backdrop-filter: blur(8px);
202     display: flex;
203     flex-direction: column;
204 }
```

```
205
206 .log-panel-header {
207     display: flex;
208     justify-content: space-between;
209     align-items: center;
210     padding: 10px 16px;
211     background: #0f172a;
212     border-bottom: 1px solid #334155;
213     border-radius: 16px 16px 0 0;
214     color: #94a3b8;
215     font-size: 0.8rem;
216     font-weight: 600;
217     cursor: move;
218     user-select: none;
219 }
220
221 .log-panel-header i {
222     color: #38bdf8;
223 }
224
225 .log-panel-content {
226     flex: 1;
227     overflow-y: auto;
228     padding: 12px 16px;
229     white-space: pre-wrap;
230     word-break: break-word;
231     font-size: 0.78rem;
232     line-height: 1.5;
233 }
234
235 /* Estilização da barra de rolagem do painel de logs */
236 .log-panel-content::-webkit-scrollbar {
237     width: 6px;
238 }
239
240 .log-panel-content::-webkit-scrollbar-track {
241     background: #1e293b;
242 }
243
244 .log-panel-content::-webkit-scrollbar-thumb {
245     background: #475569;
246     border-radius: 3px;
247 }
248
249 /* Alça de redimensionamento do painel de logs */
250 .log-panel-resize {
251     position: absolute;
252     top: 0;
253     left: 0;
254     width: 8px;
255     height: 100%;
256     cursor: ew-resize;
257     z-index: 1;
258 }
259
260 .log-actions {
261     display: flex;
262     gap: 8px;
263     align-items: center;
264 }
265
266 .log-action-btn {
267     background: none;
268     border: none;
269     color: #94a3b8;
270     cursor: pointer;
271     padding: 2px 6px;
272     border-radius: 4px;
273     font-size: 0.85rem;
```

```
274     transition: all 0.2s;
275 }
276
277 .log-action-btn:hover {
278     color: #e2e8f0;
279     background: #334155;
280 }
281
282 /* Container para notificações toast */
283 .toast-container {
284     position: fixed;
285     top: 20px;
286     right: 20px;
287     z-index: 9999;
288 }
289
290 .toast {
291     background: white;
292     border-radius: 8px;
293     padding: 12px 20px;
294     margin-bottom: 8px;
295     box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
296     animation: slideIn 0.3s ease-out;
297     display: flex;
298     align-items: center;
299     gap: 8px;
300 }
301
302 .toast.success {
303     border-left: 4px solid #28a745;
304 }
305
306 .toast.error {
307     border-left: 4px solid #dc3545;
308 }
309
310 .toast.warning {
311     border-left: 4px solid #ffc107;
312 }
313
314 @keyframes slideIn {
315     from { transform: translateX(100%); opacity: 0; }
316     to { transform: translateX(0); opacity: 1; }
317 }
318
319 @keyframes fadeOut {
320     from { opacity: 1; }
321     to { opacity: 0; }
322 }
323
324 .speed-value {
325     min-width: 60px;
326     text-align: center;
327     font-weight: 600;
328     background: #212529;
329     color: #f8f9fa;
330     padding: 4px 8px;
331     border-radius: 20px;
332 }
333 </style>
334 </head>
335 <body>
336     <!-- Container para notificações toast (mensagens temporárias) -->
337     <div class="toast-container" id="toastContainer"></div>
338
339     <!-- Botão flutuante para abrir/fechar o painel lateral -->
340     <button class="fab-button" type="button" data-bs-toggle="offcanvas" data-bs-target=
341         "#graphSidebar"
342         title="Abrir painel (Ctrl+B)">
```

```
342     <i class="bi bi-layout-sidebar-inset"></i>
343 </button>
344
345 <!-- Painel lateral com todas as ferramentas e configurações -->
346 <div class="offcanvas offcanvas-start" data-bs-scroll="true" tabindex="-1" id="graphSidebar"
347 >
348     <div class="offcanvas-header border-bottom">
349         <h5 class="offcanvas-title text-primary fw-bold">
350             <i class="bi bi-diagram-3 me-2"></i>
351             Grifon - Grafo Interativo
352             <span class="badge bg-secondary ms-2" id="badgeStats">V:0 E:0</span>
353         </h5>
354         <button type="button" class="btn-close" data-bs-dismiss="offcanvas"></button>
355     </div>
356     <div class="offcanvas-body">
357
358         <!-- =====
359             CARD DE ESTATÍSTICAS - Mostra dados em tempo real do grafo
360             ===== -->
361         <div class="info-card">
362             <div class="d-flex justify-content-between">
363                 <div>
364                     <span class="small text-muted">VÉRTICES</span><br>
365                     <span id="infoVertices" class="value">0</span>
366                 </div>
367                 <div>
368                     <span class="small text-muted">ARESTAS</span><br>
369                     <span id="infoArestas" class="value">0</span>
370                 </div>
371                 <div>
372                     <span class="small text-muted">DENSIDADE</span><br>
373                     <span id="infoDensidade" class="value">0.00</span>
374                 </div>
375             </div>
376             <div class="d-flex justify-content-between mt-2">
377                 <div>
378                     <span class="small text-muted">GRAU MÉDIO</span><br>
379                     <span id="infoGrauMedio" class="value">0.00</span>
380                 </div>
381                 <div>
382                     <span class="small text-muted">COMPONENTES</span><br>
383                     <span id="infoComponentes" class="value">0</span>
384                 </div>
385             </div>
386         </div>
387
388         <!-- Configurações do tipo de grafo -->
389         <div class="row g-2 mb-3">
390             <div class="col-6">
391                 <label class="form-label small-label fw-bold">Tipo</label>
392                 <select class="form-select form-select-sm" id="selTipo">
393                     <option value="naoorientado">Não orientado</option>
394                     <option value="orientado">Orientado</option>
395                 </select>
396             </div>
397             <div class="col-6">
398                 <label class="form-label small-label fw-bold">Peso</label>
399                 <select class="form-select form-select-sm" id="selPeso">
400                     <option value="nao">Sem peso</option>
401                     <option value="sim">Com peso</option>
402                 </select>
403             </div>
404         </div>
405
406         <!-- Opções de alternância (switches) -->
407         <div class="mb-3">
408             <div class="form-check form-switch">
409                 <input class="form-check-input" type="checkbox" id="swLoopAllowed">
410                 <label class="form-check-label small-label">Permitir loops</label>
411             </div>
412         </div>
413     </div>
414 </div>
```

```

410     </div>
411     <div class="form-check form-switch mt-1">
412         <input class="form-check-input" type="checkbox" id="swCurvedEdges" checked>
413         <label class="form-check-label small-label">
414             <i class="bi bi-bezier2 me-1"></i> Arestas curvas
415         </label>
416     </div>
417     <div class="form-check form-switch mt-1">
418         <input class="form-check-input" type="checkbox" id="swAutoFocus">
419         <label class="form-check-label small-label">Centralizar no vértice atual</label>
420     </div>
421 </div>
422
423 <hr>
424
425 <!-- Criação de vértices em lote -->
426 <div class="mb-3">
427     <div class="fw-bold small mb-2">Vértices em lote</div>
428     <div class="input-group input-group-sm mb-2">
429         <input id="inpBatchN" type="number" class="form-control" min="1" max="200" value="5"
430             title="Quantidade de vértices a criar">
431         <select id="selBatchTipo" class="form-select" title="Tipo de nomeação dos vértices">
432             <option value="alfabetico">A,B,C...</option>
433             <option value="numerico">0,1,2...</option>
434             <option value="prefixo">Prefixo</option>
435         </select>
436         <input id="inpPrefixo" type="text" class="form-control" value="V" style=
437             "max-width:70px"
438             title="Prefixo para nomeação personalizada">
439         <button id="btnBatchAdd" class="btn btn-success" title="Criar vértices em lote">
440             <i class="bi bi-plus-circle"></i>
441         </button>
442     </div>
443 <!-- Criação/remoção de vértice individual -->
444 <div class="input-group input-group-sm">
445     <input id="inpV" type="text" class="form-control" placeholder="ID vértice"
446         title="Identificador do vértice">
447     <button id="btnAddV" class="btn btn-outline-primary" title="Adicionar vértice">
448         <i class="bi bi-plus"></i>
449     </button>
450     <button id="btnRemV" class="btn btn-outline-danger" title="Remover vértice">
451         <i class="bi bi-dash"></i>
452     </button>
453 </div>
454 </div>
455
456 <hr>
457
458 <!-- Geração automática de arestas -->
459 <div class="mb-3">
460     <div class="fw-bold small mb-2">Arestas automáticas</div>
461     <div class="row g-2">
462         <div class="col-6">
463             <select class="form-select form-select-sm" id="selAutoEdgesMode"
464                 title="Modo de geração automática de arestas">
465                 <option value="caminho">Caminho</option>
466                 <option value="ciclo">Ciclo</option>
467                 <option value="completo">Completo</option>
468                 <option value="aleatorio">Aleatório</option>
469             </select>
470         </div>
471         <div class="col-6">
472             <input id="inpAutoEdgesM" type="number" class="form-control form-control-sm"
473                 min="0" value="6" placeholder="Qtd" title="Quantidade para modo aleatório">
474         </div>
475         <div class="col-12 d-flex gap-2">
476             <button id="btnAutoEdges" class="btn btn-outline-success btn-sm flex-grow-1"
477                 title="Gerar arestas automaticamente">
478                 <i class="bi bi-shuffle"></i> Gerar

```

```

478         </button>
479         <button id="btnAutoEdgesClear" class="btn btn-outline-danger btn-sm"
480             title="Remover todas as arestas">
481             <i class="bi bi-x-circle"></i>
482         </button>
483     </div>
484 </div>
485 </div>
486
487 <hr>
488
489 <!-- Criação de aresta individual -->
490 <div class="mb-3">
491     <div class="fw-bold small mb-2">Aresta individual</div>
492     <div class="input-group input-group-sm mb-2">
493         <input id="inpU" type="text" class="form-control" placeholder="de"
494             title="Vértice de origem">
495         <span class="input-group-text">→</span>
496         <input id="inpW" type="text" class="form-control" placeholder="para"
497             title="Vértice de destino">
498         <input id="inpPeso" type="number" class="form-control" placeholder="peso"
499             style="max-width:80px" title="Peso da aresta (opcional)">
500     </div>
501     <div class="d-flex gap-2">
502         <button id="btnAddE" class="btn btn-outline-success btn-sm flex-grow-1"
503             title="Adicionar aresta">
504             <i class="bi bi-plus"></i> Adicionar
505         </button>
506         <button id="btnRemE" class="btn btn-outline-danger btn-sm flex-grow-1"
507             title="Remover aresta">
508             <i class="bi bi-dash"></i> Remover
509         </button>
510     </div>
511 </div>
512
513 <hr>
514
515 <!-- Seção de coloração do grafo -->
516 <div class="mb-2">
517     <div class="fw-bold small">Coloração Completa</div>
518     <div class="d-flex gap-2 mt-2">
519         <button id="btnColorirVertices" class="btn btn-outline-primary btn-sm flex-grow-1"
520             title="Colorir apenas os vértices (Algoritmo Welsh-Powell)">
521             <i class="bi bi-palette"></i> Colorir Vértices
522         </button>
523         <button id="btnColorirArestas" class="btn btn-outline-success btn-sm flex-grow-1"
524             title="Colorir apenas as arestas (Algoritmo Guloso ou Round-Robin)">
525             <i class="bi bi-palette-fill"></i> Colorir Arestas
526         </button>
527     </div>
528     <div class="d-flex gap-2 mt-2">
529         <button id="btnColorirTudo" class="btn btn-outline-warning btn-sm flex-grow-1"
530             title="Colorir vértices e arestas simultaneamente">
531             <i class="bi bi-palette2"></i> Colorir Tudo
532         </button>
533         <button id="btnColorirReset" class="btn btn-outline-secondary btn-sm"
534             title="Voltar às cores padrão">
535             <i class="bi bi-eraser"></i> Resetar Cores
536         </button>
537     </div>
538     <div class="d-flex gap-2 mt-2">
539         <button id="btnColorirAnimado" class="btn btn-outline-info btn-sm flex-grow-1"
540             title="Assistir à coloração passo a passo">
541             <i class="bi bi-play-circle"></i> 🎬 Colorir (Passo a Passo)
542         </button>
543     </div>
544 </div>
545
546 <hr>

```



```

547
548 <!-- Controle de velocidade da animação -->
549 <div class="mb-3">
550   <div class="fw-bold small mb-2">Velocidade da Animação</div>
551   <div class="d-flex align-items-center gap-2">
552     <button id="btnSpeedDown" class="btn btn-outline-secondary btn-sm"
553       title="Diminuir velocidade">-</button>
554     <span id="speedValue" class="badge bg-dark" style="min-width:60px">1050</span>
555     <span class="small">ms</span>
556     <button id="btnSpeedUp" class="btn btn-outline-secondary btn-sm"
557       title="Aumentar velocidade">+</button>
558     <button id="btnSpeedReset" class="btn btn-outline-secondary btn-sm"
559       title="Velocidade padrão">↺</button>
560   </div>
561 </div>
562
563 <hr>
564
565 <!-- Modelos de grafos pré-definidos -->
566 <div class="mb-2">
567   <div class="fw-bold small mb-2">Modelos pré-definidos</div>
568   <div class="input-group input-group-sm">
569     <select id="selModelo" class="form-select" title="Selecione um modelo de grafo">
570       <option value="vazio">Grafo vazio</option>
571       <option value="completo5">Grafo completo K5</option>
572       <option value="caminho5">Caminho P5</option>
573       <option value="ciclo5">Ciclo C5</option>
574       <option value="bipartido">Bipartido K3,3</option>
575       <option value="margaridas">Grafo Margaridas</option>
576       <option value="simples">Simples (quadrado com diagonal)</option>
577     </select>
578     <button id="btnCarregarModelo" class="btn btn-primary" title="Carregar modelo
579       selecionado">
580       <i class="bi bi-database"></i> Carregar
581     </button>
582   </div>
583 </div>
584
585 <hr>
586
587 <!-- Operações de arquivo (salvar/carregar) -->
588 <div class="mb-2">
589   <div class="fw-bold small mb-2">Arquivo .txt</div>
590   <div class="input-group input-group-sm">
591     <input id="inpFileName" type="text" class="form-control" value="grafo.txt"
592       title="Nome do arquivo para salvar">
593     <button id="btnSaveFile" class="btn btn-primary" title="Salvar grafo em arquivo">
594       <i class="bi bi-download"></i>
595     </button>
596     <input id="fileInput" type="file" class="form-control" accept=".txt" style=
597       "display:none">
598     <button id="btnLoadFile" class="btn btn-secondary" title="Carregar grafo de arquivo">
599       <i class="bi bi-upload"></i>
600     </button>
601   </div>
602 </div>
603
604 <hr>
605
606 <!-- Ações gerais -->
607 <div class="d-grid gap-2 mt-3">
608   <button class="btn btn-outline-dark btn-sm" id="btnStabilize"
609     title="Centralizar todo o grafo na tela">
610     <i class="bi bi-magic"></i> Centralizar
611   </button>
612   <button class="btn btn-outline-warning btn-sm" id="btnResetColors"
613     title="Limpar cores de destaque">
614     <i class="bi bi-palette"></i> Limpar destaque

```

```

613         </button>
614         <button class="btn btn-outline-danger btn-sm" id="btnUndo"
615             title="Desfazer última ação (Ctrl+Z)">
616             <i class="bi bi-arrow-counterclockwise"></i> Desfazer
617         </button>
618         <button class="btn btn-outline-danger btn-sm" id="btnRedo"
619             title="Refazer ação desfeita (Ctrl+Y)">
620             <i class="bi bi-arrow-clockwise"></i> Refazer
621         </button>
622         <button class="btn btn-outline-danger btn-sm" id="btnClear"
623             title="Limpar todo o grafo">
624             <i class="bi bi-trash"></i> Limpar tudo
625         </button>
626     </div>
627 </div>
628 </div>
629
630 <!-- Área principal da aplicação -->
631 <div class="container-fluid py-2" style="position: relative;">
632     <div class="d-flex align-items-center justify-content-end mt-5 mb-2">
633         <h5 class="m-0 text-primary me-auto">
634             <i class="bi bi-diagram-3 me-2"></i>
635             Grifon - Editor de Grafos
636             <span class="badge bg-secondary ms-2" id="badgeStatsTop">V:0 E:0</span>
637         </h5>
638     </div>
639     <div class="grifon-bg"></div>
640
641     <div class="position-relative">
642         <!-- Barra de ferramentas flutuante -->
643         <div class="toolbar-stage">
644             <button id="modeAddVertex" class="btn btn-sm btn-outline-primary"
645                 title="Modo: Adicionar vértices ao clicar na área">
646                 <i class="bi bi-node-plus"></i> Vértice
647             </button>
648             <button id="modeAddEdge" class="btn btn-sm btn-outline-success"
649                 title="Modo: Criar arestas entre vértices">
650                 <i class="bi bi-bezier2"></i> Aresta
651             </button>
652             <button id="modeSelect" class="btn btn-sm btn-outline-secondary mode-active"
653                 title="Modo: Selecionar e mover elementos">
654                 <i class="bi bi-cursor"></i> Selecionar
655             </button>
656         </div>
657
658         <!-- Container do grafo Cytoscape -->
659         <div class="card shadow-sm border-0">
660             <div class="card-body p-0">
661                 <div id="cy"></div>
662
663                 <!-- Painel de logs (console da aplicação) -->
664                 <div class="log-panel" id="logPanel">
665                     <div class="log-panel-resize" id="logResize"></div>
666                     <div class="log-panel-header" id="logHeader">
667                         <span><i class="bi bi-terminal"></i> Console de Logs</span>
668                         <div class="log-actions">
669                             <button class="log-action-btn" id="btnClearLog" title="Limpar logs">
670                                 <i class="bi bi-eraser"></i>
671                             </button>
672                             <button class="log-action-btn" id="btnToggleLog" title="Minimizar/Maximizar">
673                                 <i class="bi bi-chevron-up"></i>
674                             </button>
675                             <span id="logTimestamp" style="font-size:0.7rem;">--:--:--</span>
676                         </div>
677                     </div>
678                     <div class="log-panel-content" id="logPanelContent"></div>
679                 </div>
680             </div>
681         </div>

```

```

682     </div>
683
684     <!-- Rodapé com informações de versão -->
685     <div class="mt-2">
686         <div class="fw-bold small mb-1">
687             <div class="border rounded-pill px-3 py-1 bg-light">
688                 <i class="bi bi-tag-fill text-primary"></i>
689                 <span id="versaoDisplay">v3.6.0-cytoscape</span>
690                 <i class="bi bi-calendar3 ms-2 text-primary"></i> |
691                 <span>2026</span> |
692                 <span>Por Jailis Dourado</span>
693             </div>
694         </div>
695     </div>
696 </div>
697
698 <!-- Menu de contexto (aparece com clique direito) -->
699 <div id="ctxMenu" class="ctx-menu">
700     <button data-action="add-node">
701         <i class="bi bi-plus-circle me-2"></i>Adicionar vértice aqui
702     </button>
703     <button data-action="rename-node">
704         <i class="bi bi-pencil-square me-2"></i>Renomear vértice
705     </button>
706     <button data-action="delete-node">
707         <i class="bi bi-trash me-2"></i>Excluir vértice
708     </button>
709     <div class="divider"></div>
710     <button data-action="start-edge">
711         <i class="bi bi-bezier2 me-2"></i>Iniciar aresta
712     </button>
713     <button data-action="edit-edge">
714         <i class="bi bi-sliders me-2"></i>Editar peso
715     </button>
716     <button data-action="delete-edge">
717         <i class="bi bi-x-circle me-2"></i>Excluir aresta
718     </button>
719     <button data-action="cancel-edge">
720         <i class="bi bi-backspace me-2"></i>Cancelar criação
721     </button>
722 </div>
723
724 <script>
725     // =====
726     // INÍCIO DO CÓDIGO PRINCIPAL
727     // Tudo está encapsulado em uma IIFE (Immediately Invoked Function Expression)
728     // para evitar poluir o escopo global e evitar conflitos com outros scripts
729     // =====
730     (function() {
731         'use strict'; // Modo estrito: ajuda a evitar erros comuns de programação
732
733         // =====
734         // CONSTANTES DO SISTEMA
735         // Estas são configurações que não mudam durante a execução
736         // =====
737         const VERSAO = '3.6.0'; // Versão atual do Grifon
738         const MAXIMO_VERTICES = 200; // Limite máximo de vértices no grafo
739
740         // Modos de operação da ferramenta (o que acontece ao clicar no grafo)
741         const MODOS = Object.freeze({
742             SELECIONAR: 'select', // Modo padrão: selecionar e mover elementos
743             VERTICE: 'vertex', // Modo vértice: clicar cria novo vértice
744             ARESTA: 'edge' // Modo aresta: clicar em dois vértices cria aresta
745         });
746
747         // Tipos de grafo suportados
748         const TIPOS_GRAFO = Object.freeze({
749             ORIENTADO: 'orientado', // Arestas têm direção (setas)
750             NAO_ORIENTADO: 'naoorientado' // Arestas não têm direção

```

```

751     });
752
753     // Tipos de notificações toast
754     const NOTIFICACAO = Object.freeze({
755         SUCESSO: 'success',    // Verde - operação bem sucedida
756         ERRO: 'error',        // Vermelho - algo deu errado
757         AVISO: 'warning'      // Amarelo - atenção necessária
758     });
759
760     // =====
761     // PALETAS DE CORES
762     // Conjuntos de cores pré-definidas para coloração de vértices e arestas
763     // =====
764
765     // 20 cores para vértices - escolhidas para serem visualmente distintas
766     const PALETA_VERTICES = [
767         '#E63946', '#2A9D8F', '#2874A6', '#F4A261', '#9B59B6',
768         '#1ABC9C', '#E67E22', '#C0392B', '#16A085', '#2980B9',
769         '#F1C40F', '#8E44AD', '#27AE60', '#D35400', '#3498DB',
770         '#E91E63', '#00BCD4', '#FF5722', '#673AB7', '#4CAF50'
771     ];
772
773     // 22 cores para arestas - cores vibrantes e diferenciáveis
774     const PALETA_ARESTAS = [
775         '#E63946', '#2A9D8F', '#2874A6', '#F4A261', '#9B59B6',
776         '#1ABC9C', '#E67E22', '#C0392B', '#16A085', '#2980B9',
777         '#F1C40F', '#8E44AD', '#27AE60', '#D35400', '#3498DB',
778         '#E91E63', '#00BCD4', '#FF5722', '#673AB7', '#4CAF50',
779         '#FF6B35', '#5D576B'
780     ];
781
782     // =====
783     // FUNÇÕES UTILITÁRIAS (Helpers)
784     // Funções de uso geral que simplificam o código
785     // =====
786
787     /**
788      * Atalho para document.getElementById()
789      * @param {string} id - O ID do elemento HTML
790      * @returns {HTMLElement} O elemento encontrado
791      */
792     const pegarElemento = (id) => document.getElementById(id);
793
794     // Referências aos elementos de log
795     const caixaLog = pegarElemento('log') || document.createElement('div');
796     const painelLogConteudo = pegarElemento('logPanelContent');
797     const timestampLog = pegarElemento('logTimestamp');
798
799     /**
800      * Exibe uma notificação toast temporária na tela
801      * @param {string} mensagem - Texto da notificação
802      * @param {string} tipo - Tipo: 'success', 'error' ou 'warning'
803      */
804     function exibirNotificacao(mensagem, tipo = NOTIFICACAO.SUCESSO) {
805         const container = pegarElemento('toastContainer');
806         const toast = document.createElement('div');
807         toast.className = `toast ${tipo}`;
808
809         // Define o ícone baseado no tipo da notificação
810         const icone = tipo === NOTIFICACAO.SUCESSO ? 'check-circle' :
811             tipo === NOTIFICACAO.ERRO ? 'exclamation-circle' :
812             'info-circle';
813
814         toast.innerHTML = `
815             <i class="bi bi-${icone}"></i>
816             <span>${mensagem}</span>
817         `;
818         container.appendChild(toast);
819

```

```

820 // Remove a notificação após 3 segundos com animação
821 setTimeout(() => {
822     toast.style.animation = 'fadeOut 0.3s ease-out forwards';
823     setTimeout(() => toast.remove(), 300);
824 }, 3000);
825 }
826
827 /**
828  * Registra uma mensagem no console de Logs
829  * @param {string} mensagem - Texto a ser registrado
830  */
831 function registrarLog(mensagem) {
832     const horario = new Date().toLocaleTimeString();
833     const entradaLog = `[${horario}] ${mensagem}`;
834
835     // Adiciona ao elemento de log oculto (se existir)
836     if (caixaLog && caixaLog !== document.createElement('div')) {
837         caixaLog.textContent += entradaLog + '\n';
838     }
839
840     // Adiciona ao painel de log visível
841     if (painelLogConteudo) {
842         painelLogConteudo.textContent += entradaLog + '\n';
843         // Rolagem automática para a última mensagem
844         painelLogConteudo.scrollTop = painelLogConteudo.scrollHeight;
845     }
846
847     // Atualiza o timestamp no cabeçalho do painel
848     if (timestampLog) {
849         timestampLog.textContent = horario;
850     }
851 }
852
853 /**
854  * Calcula a cor de texto ideal (branca ou preta) baseada na cor de fundo
855  * Usa a fórmula de luminância relativa para garantir contraste adequado
856  *
857  * @param {string} corHexadecimal - Cor no formato #RRGGBB
858  * @returns {string} '#ffffff' para fundos escuros ou '#000000' para fundos claros
859  */
860 function obterCorContraste(corHexadecimal) {
861     if (!corHexadecimal || !corHexadecimal.startsWith('#')) return '#ffffff';
862
863     // Extraí os componentes RGB da cor hexadecimal
864     const r = parseInt(corHexadecimal.slice(1, 3), 16);
865     const g = parseInt(corHexadecimal.slice(3, 5), 16);
866     const b = parseInt(corHexadecimal.slice(5, 7), 16);
867
868     // Calcula a luminância percebida (fórmula ITU-R BT.709)
869     // 0.299*R + 0.587*G + 0.114*B
870     const luminancia = (0.299 * r + 0.587 * g + 0.114 * b) / 255;
871
872     // Se a cor de fundo for clara, usa texto preto; se escura, texto branco
873     return luminancia > 0.5 ? '#000000' : '#ffffff';
874 }
875
876 // =====
877 // GERENCIADOR DE HISTÓRICO (UNDO/REDO)
878 // Implementa o padrão Command para desfazer e refazer ações
879 // Mantém duas pilhas: uma para desfazer, outra para refazer
880 // =====
881 class GerenciadorHistorico {
882     constructor() {
883         this.pilhaDesfazer = []; // Pilha de ações que podem ser desfeitas
884         this.pilhaRefazer = []; // Pilha de ações que podem ser refeitas
885         this.tamanhoMaximo = 50; // Limite de ações armazenadas
886     }
887
888     /**

```

```

889      * Adiciona uma nova ação ao histórico
890      * @param {Object} acao - Objeto com funções undo() e redo()
891      */
892      adicionar(acao) {
893          this.pilhaDesfazer.push({
894              ...acao,
895              timestamp: Date.now()
896          });
897
898          // Remove a ação mais antiga se exceder o limite
899          if (this.pilhaDesfazer.length > this.tamanhoMaximo) {
900              this.pilhaDesfazer.shift();
901          }
902
903          // Limpa a pilha de refazer quando uma nova ação é executada
904          this.pilhaRefazer = [];
905      }
906
907      /**
908       * Desfaz a última ação
909       * @returns {Object|null} A ação desfeita ou null se não houver ações
910       */
911      desfazer() {
912          if (this.pilhaDesfazer.length === 0) return null;
913          const acao = this.pilhaDesfazer.pop();
914          this.pilhaRefazer.push(acao);
915          return acao;
916      }
917
918      /**
919       * Refaz a última ação desfeita
920       * @returns {Object|null} A ação refeita ou null se não houver ações
921       */
922      refazer() {
923          if (this.pilhaRefazer.length === 0) return null;
924          const acao = this.pilhaRefazer.pop();
925          this.pilhaDesfazer.push(acao);
926          return acao;
927      }
928
929      /** Limpa todo o histórico */
930      limpar() {
931          this.pilhaDesfazer = [];
932          this.pilhaRefazer = [];
933      }
934
935      /** @returns {boolean} true se há ações para desfazer */
936      podeDesfazer() {
937          return this.pilhaDesfazer.length > 0;
938      }
939
940      /** @returns {boolean} true se há ações para refazer */
941      podeRefazer() {
942          return this.pilhaRefazer.length > 0;
943      }
944  }
945
946  // =====
947  // CLASSE PRINCIPAL DA APLICAÇÃO
948  // Contém toda a lógica do editor de grafos Grifon
949  // =====
950  class AplicacaoGrifon {
951      constructor() {
952          this.cy = null; // Instância do Cytoscape
953          this.historico = new GerenciadorHistorico(); // Gerenciador de undo/redo
954
955          // Estado atual da aplicação (todas as configurações e seleções)
956          this.estado = {
957              orientado: false, // Grafo é orientado?

```

```

958     ponderado: false, // Arestas têm peso?
959     permitirLacos: false, // Permite loops (vértice conectado a si mesmo)?
960     arestasCurvas: true, // Arestas são desenhadas curvas?
961     modo: MODOS.SELECIONAR, // Modo atual de operação
962     arestaOrigem: null, // Vértice de origem da aresta sendo criada
963     verticeSelecionadoId: null, // ID do vértice atualmente selecionado
964     arestaSelecionadaId: null, // ID da aresta atualmente selecionada
965     contadorVertices: 1, // Contador para nomeação automática V1, V2...
966     ctxVerticeId: null, // ID do vértice sob o menu de contexto
967     ctxArestaId: null, // ID da aresta sob o menu de contexto
968     ctxPosicaoTela: null, // Posição do clique para menu de contexto
969     executandoAnimacao: false, // Há uma animação em andamento?
970     idExecucaoAtual: 0, // ID da animação atual (para cancelamento)
971     velocidadeAnimacao: 1050 // Velocidade da animação em milissegundos
972 };
973
974     this.inicializar();
975 }
976
977 /**
978  * Inicialização assíncrona da aplicação
979  * Configura o Cytoscape, eventos e estado inicial
980  */
981 async inicializar() {
982     try {
983         this.inicializarCytoscape();
984         this.conectarEventosInterface();
985         this.conectarEventosCytoscape();
986         this.conectarAtalhosTeclado();
987         this.atualizarIndicadores();
988         this.definirModo(MODOS.SELECIONAR);
989         this.definirEstadoExecucao(false);
990
991         registrarLog(`🦁 Grifon v${VERSAO} inicializado com sucesso!`);
992         registrarLog(`📁 Algoritmos: Welsh-Powell (vértices) | Round-Robin para K ímpar | Guloso (arestas)`);
993         registrarLog(`👉 Clique direito para menu | Duplo clique na aresta para editar peso`);
994         registrarLog(`🗑️ Ctrl+Z: Desfazer | Ctrl+Y: Refazer | Delete: Remover | Esc: Cancelar`);
995
996         exibirNotificacao('Grifon pronto para uso!', NOTIFICACAO.SUCESSO);
997     } catch (erro) {
998         console.error('Erro na inicialização:', erro);
999         exibirNotificacao('Erro ao inicializar o Grifon', NOTIFICACAO.ERRO);
1000     }
1001 }
1002
1003 /**
1004  * Configura a instância do Cytoscape.js
1005  * Define estilos, layout e comportamento do grafo
1006  */
1007 inicializarCytoscape() {
1008     this.cy = cytoscape({
1009         container: pegarElemento('cy'), // Elemento HTML que conterá o grafo
1010
1011         // Estilos CSS-like para os elementos do grafo
1012         style: [
1013             // Estilo padrão dos vértices (nós)
1014             {
1015                 selector: 'node',
1016                 style: {
1017                     'background-color': '#a5d8ff', // Azul claro
1018                     'border-color': '#1971c2', // Borda azul escuro
1019                     'border-width': 2,
1020                     'label': 'data(id)', // Rótulo = ID do vértice
1021                     'font-size': 12,
1022                     'font-family': 'Segoe UI',
1023                     'font-weight': 'bold',

```



```

1024         'text-valign': 'center',
1025         'text-halign': 'center',
1026         'color': '#1e293b', // Texto escuro
1027         'width': 40,
1028         'height': 40,
1029         'transition-property': 'background-color, border-color, color',
1030         'transition-duration': '0.2s'
1031     }
1032 },
1033 // Vértice selecionado
1034 {
1035     selector: 'node:selected',
1036     style: {
1037         'background-color': '#74c0fc', // Azul mais claro
1038         'border-color': '#0d6efd', // Borda azul destacada
1039         'border-width': 3
1040     }
1041 },
1042 // Estilo padrão das arestas
1043 {
1044     selector: 'edge',
1045     style: {
1046         'width': 2,
1047         'line-color': '#868e96', // Cinza
1048         'target-arrow-color': '#868e96',
1049         'target-arrow-shape': 'none', // Sem seta (grafo não orientado)
1050         'curve-style': 'bezier', // Curvas de Bézier
1051         'control-point-step-size': 40,
1052         'label': 'data(label)', // Rótulo = peso da aresta
1053         'font-size': 10,
1054         'text-opacity': 0, // Rótulo oculto por padrão
1055         'transition-property': 'line-color, target-arrow-color, width',
1056         'transition-duration': '0.2s'
1057     }
1058 },
1059 // Aresta selecionada
1060 {
1061     selector: 'edge:selected',
1062     style: {
1063         'width': 3,
1064         'line-color': '#0d6efd',
1065         'target-arrow-color': '#0d6efd'
1066     }
1067 },
1068 // Aresta com rótulo visível (grafos ponderados)
1069 {
1070     selector: 'edge.show-label',
1071     style: {
1072         'text-opacity': 1,
1073         'text-background-color': 'ffffff',
1074         'text-background-opacity': 0.8,
1075         'text-background-padding': '2px'
1076     }
1077 },
1078 // Aresta pendente (sendo criada)
1079 {
1080     selector: 'edge.pendente',
1081     style: {
1082         'line-style': 'dashed',
1083         'line-color': '#ffc107' // Amarelo
1084     }
1085 }
1086 ],
1087 layout: { name: 'preset' }, // Layout preserva posições manuais
1088 wheelSensitivity: 0.5, // Sensibilidade do zoom com scroll
1089 pixelRatio: 2 // Qualidade de renderização (retina)
1090 });
1091 }
1092

```



```

1093 // =====
1094 // OPERAÇÕES BÁSICAS DO GRAFO
1095 // Métodos para adicionar, remover e modificar vértices e arestas
1096 // =====
1097
1098 /**
1099  * Adiciona um novo vértice ao grafo
1100  *
1101  * @param {string} id - Identificador único do vértice
1102  * @param {Object|null} posicao - Posição {x, y} ou null para posição aleatória
1103  * @returns {boolean} true se o vértice foi adicionado com sucesso
1104  */
1105 adicionarVertice(id, posicao = null) {
1106     // Verifica se há animação em andamento (não permite modificar durante animação)
1107     if (this.estado.executandoAnimacao) {
1108         registrarLog('⊖ Operação bloqueada durante animação');
1109         return false;
1110     }
1111
1112     id = String(id).trim();
1113
1114     // Validações do ID do vértice
1115     if (!id) {
1116         exibirNotificacao('ID do vértice inválido', NOTIFICACAO.ERRO);
1117         return false;
1118     }
1119
1120     if (this.cy.getElementById(id).length) {
1121         exibirNotificacao(`Vértice "${id}" já existe`, NOTIFICACAO.AVISO);
1122         return false;
1123     }
1124
1125     if (this.cy.nodes().length >= MAXIMO_VERTICES) {
1126         exibirNotificacao(`Limite de ${MAXIMO_VERTICES} vértices atingido`,
1127             NOTIFICACAO.ERRO);
1128         return false;
1129     }
1130
1131     // Define posição: usa a fornecida ou gera uma aleatória
1132     const pos = (posicao && Number.isFinite(posicao.x) && Number.isFinite(posicao.y))
1133         ? posicao
1134         : { x: 300 + Math.random() * 200, y: 200 + Math.random() * 200 };
1135
1136     // Adiciona o vértice ao Cytoscape
1137     this.cy.add({ group: 'nodes', data: { id } }).position(pos);
1138
1139     // Salva no histórico para permitir desfazer
1140     this.historico.adicionar({
1141         tipo: 'adicionar-vertice',
1142         dados: { id, pos: { ...pos } },
1143         desfazer: () => this.removerVertice(id, false),
1144         refazer: () => this.adicionarVertice(id, pos)
1145     });
1146
1147     this.atualizarIndicadores();
1148     registrarLog(`☑ Vértice "${id}" adicionado`);
1149     exibirNotificacao(`Vértice "${id}" criado`, NOTIFICACAO.SUCESSO);
1150     return true;
1151 }
1152
1153 /**
1154  * Renomeia um vértice existente
1155  *
1156  * @param {string} idAntigo - ID atual do vértice
1157  * @param {string} idNovo - Novo ID para o vértice
1158  * @returns {boolean} true se renomeado com sucesso
1159  */
1160 renomearVertice(idAntigo, idNovo) {
1161     if (this.estado.executandoAnimacao) {

```

```

1161     registrarLog('⊖ Operação bloqueada durante animação');
1162     return false;
1163 }
1164
1165 const no = this.cy.getElementById(idAntigo);
1166 if (!no.length) {
1167     exibirNotificacao(`Vértice "${idAntigo}" não encontrado`, NOTIFICACAO.ERRO);
1168     return false;
1169 }
1170
1171 idNovo = String(idNovo).trim();
1172
1173 if (!idNovo) {
1174     exibirNotificacao('Novo ID inválido', NOTIFICACAO.ERRO);
1175     return false;
1176 }
1177
1178 if (this.cy.getElementById(idNovo).length) {
1179     exibirNotificacao(`Já existe vértice "${idNovo}"`, NOTIFICACAO.ERRO);
1180     return false;
1181 }
1182
1183 // Guarda informações das arestas para o histórico
1184 const arestasConectadas = no.connectedEdges();
1185 const arestasAntigas = arestasConectadas.map(e => ({
1186     id: e.id(),
1187     origem: e.data('source'),
1188     destino: e.data('target'),
1189     rotulo: e.data('label')
1190 }));
1191
1192 // Executa a renomeação
1193 no.data('id', idNovo);
1194 arestasConectadas.forEach(e => {
1195     if (e.data('source') === idAntigo) e.data('source', idNovo);
1196     if (e.data('target') === idAntigo) e.data('target', idNovo);
1197     e.style('label', e.data('label'));
1198 });
1199 no.style('label', idNovo);
1200
1201 // Salva no histórico
1202 this.historico.adicionar({
1203     tipo: 'renomear-vertice',
1204     dados: { idAntigo, idNovo, arestas: arestasAntigas },
1205     desfazer: () => this.renomearVertice(idNovo, idAntigo),
1206     refazer: () => this.renomearVertice(idAntigo, idNovo)
1207 });
1208
1209 this.atualizarIndicadores();
1210 registrarLog(`✍ Vértice "${idAntigo}" renomeado para "${idNovo}"`);
1211 exibirNotificacao(`Vértice renomeado para "${idNovo}"`, NOTIFICACAO.SUCESSO);
1212 return true;
1213 }
1214
1215 /**
1216  * Adiciona uma aresta entre dois vértices
1217  *
1218  * @param {string} origem - ID do vértice de origem
1219  * @param {string} destino - ID do vértice de destino
1220  * @param {number|null} peso - Peso da aresta (opcional)
1221  * @param {boolean} exibirLog - Se deve registrar no console
1222  * @returns {boolean} true se adicionada com sucesso
1223  */
1224 adicionarAresta(origem, destino, peso = null, exibirLog = true) {
1225     if (this.estado.executandoAnimacao) {
1226         registrarLog('⊖ Operação bloqueada durante animação');
1227         return false;
1228     }
1229

```

```

1230     origem = String(origem).trim();
1231     destino = String(destino).trim();
1232
1233     // Verifica se os vértices existem
1234     if (!this.cy.getElementById(origem).length) {
1235         exibirNotificacao(`Vértice "${origem}" não existe`, NOTIFICACAO.ERRO);
1236         return false;
1237     }
1238
1239     if (!this.cy.getElementById(destino).length) {
1240         exibirNotificacao(`Vértice "${destino}" não existe`, NOTIFICACAO.ERRO);
1241         return false;
1242     }
1243
1244     // Verifica se é um loop e se loops são permitidos
1245     if (origem === destino && !this.estado.permitirLacos) {
1246         exibirNotificacao('Loops não permitidos. Ative no painel.', NOTIFICACAO.AVISO);
1247         return false;
1248     }
1249
1250     // Verifica arestas duplicadas (apenas para grafos orientados)
1251     const arestaDuplicada = this.cy.edges().filter(e =>
1252         (e.data('source') === origem && e.data('target') === destino) ||
1253         (!this.estado.orientado && e.data('source') === destino && e.data('target') ===
1254             origem)
1255     );
1256
1257     if (arestaDuplicada.length && this.estado.orientado) {
1258         exibirNotificacao(`Aresta ${origem}→${destino} já existe`, NOTIFICACAO.AVISO);
1259         return false;
1260     }
1261
1262     // Define o valor do peso (se aplicável)
1263     const valorPeso = this.estado.ponderado && peso !== null && peso !== '' && !isNaN(
1264         Number(peso))
1265         ? String(peso)
1266         : '';
1267
1268     // Gera ID único para a aresta
1269     const idAresta =
1270         `e_${origem}_${destino}_${Date.now()}_${Math.random().toString(36).slice(2, 7)}`;
1271
1272     // Adiciona a aresta
1273     this.cy.add({
1274         group: 'edges',
1275         data: {
1276             id: idAresta,
1277             source: origem,
1278             target: destino,
1279             label: valorPeso
1280         }
1281     });
1282
1283     // Salva no histórico
1284     this.historico.adicionar({
1285         tipo: 'adicionar-aresta',
1286         dados: { id: idAresta, origem, destino, rotulo: valorPeso },
1287         desfazer: () => this.removerArestaPorId(idAresta),
1288         refazer: () => this.adicionarAresta(origem, destino, peso, false)
1289     });
1290
1291     this.atualizarEstiloArestas();
1292     this.atualizarIndicadores();
1293
1294     if (exibirLog) {
1295         registrarLog(`☑ Aresta ${origem} → ${destino}${valorPeso ? ` (peso: ${valorPeso})` : ''} adicionada`);
1296         exibirNotificacao('Aresta criada com sucesso', NOTIFICACAO.SUCESSO);
1297     }

```

```

1295
1296     return true;
1297 }
1298
1299 /**
1300  * Remove um vértice e todas as arestas conectadas a ele
1301  *
1302  * @param {string} id - ID do vértice a remover
1303  * @param {boolean} confirmar - Se deve pedir confirmação ao usuário
1304  * @returns {boolean} true se removido com sucesso
1305  */
1306 removerVertice(id, confirmar = true) {
1307     if (this.estado.executandoAnimacao) {
1308         registrarLog('⊖ Operação bloqueada durante animação');
1309         return false;
1310     }
1311
1312     id = String(id).trim();
1313     const no = this.cy.getElementById(id);
1314
1315     if (!no.length) {
1316         exibirNotificacao(`Vértice "${id}" não existe`, NOTIFICACAO.ERRO);
1317         return false;
1318     }
1319
1320     if (confirmar && !confirm(`Remover vértice "${id}" e suas arestas?`)) {
1321         return false;
1322     }
1323
1324     // Guarda estado para o histórico
1325     const posicao = { ...no.position() };
1326     const arestasConectadas = no.connectedEdges().map(e => ({
1327         id: e.id(),
1328         origem: e.data('source'),
1329         destino: e.data('target'),
1330         rotulo: e.data('label')
1331     })));
1332
1333     no.remove();
1334
1335     // Salva no histórico com função de restauração completa
1336     this.historico.adicionar({
1337         tipo: 'remover-vertice',
1338         dados: { id, posicao, arestas: arestasConectadas },
1339         desfazer: () => {
1340             this.adicionarVertice(id, posicao);
1341             arestasConectadas.forEach(aresta => {
1342                 if (this.cy.getElementById(aresta.origem).length &&
1343                     this.cy.getElementById(aresta.destino).length) {
1344                     this.adicionarAresta(aresta.origem, aresta.destino, aresta.rotulo, false);
1345                 }
1346             });
1347         },
1348         refazer: () => this.removerVertice(id, false)
1349     });
1350
1351     this.atualizarIndicadores();
1352     registrarLog(`☒ Vértice "${id}" removido`);
1353     exibirNotificacao(`Vértice "${id}" excluído`, NOTIFICACAO.SUCESSO);
1354     return true;
1355 }
1356
1357 /**
1358  * Remove uma aresta pelo seu ID
1359  *
1360  * @param {string} idAresta - ID da aresta a remover
1361  * @returns {boolean} true se removida com sucesso
1362  */
1363 removerArestaPorId(idAresta) {

```

```

1364     const aresta = this.cy.getElementById(idAresta);
1365     if (!aresta.length) return false;
1366
1367     const dadosAresta = {
1368         id: idAresta,
1369         origem: aresta.data('source'),
1370         destino: aresta.data('target'),
1371         rotulo: aresta.data('label')
1372     };
1373
1374     aresta.remove();
1375
1376     // Salva no histórico
1377     this.historico.adicionar({
1378         tipo: 'remover-aresta',
1379         dados: dadosAresta,
1380         desfazer: () => this.adicionarAresta(dadosAresta.origem, dadosAresta.destino,
1381             dadosAresta.rotulo, false),
1382         refazer: () => this.removerArestaPorId(idAresta)
1383     });
1384
1385     this.atualizarIndicadores();
1386     registrarLog('🗑️ Aresta removida');
1387     return true;
1388 }
1389
1390 /**
1391  * Remove todos os elementos do grafo
1392  * Mantém estado para permitir desfazer
1393  */
1394 limparTudo() {
1395     if (this.estado.executandoAnimacao) {
1396         registrarLog('🚫 Operação bloqueada durante animação');
1397         return;
1398     }
1399
1400     if (!confirm('Tem certeza que deseja limpar todo o grafo?')) {
1401         return;
1402     }
1403
1404     // Exporta o estado atual para permitir restauração
1405     const estadoCompleto = this.exportarParaTexto();
1406
1407     this.cy.elements().remove();
1408     this.estado.contadorVertices = 1;
1409     this.limparCoresBusca();
1410
1411     this.historico.adicionar({
1412         tipo: 'limpar-tudo',
1413         dados: { estado: estadoCompleto },
1414         desfazer: () => this.importarDeTexto(estadoCompleto),
1415         refazer: () => this.limparTudo()
1416     });
1417
1418     this.atualizarIndicadores();
1419     registrarLog('🗑️ Grafo completamente limpo');
1420     exibirNotificacao('Grafo limpo com sucesso', NOTIFICACAO.SUCESSO);
1421 }
1422
1423 // =====
1424 // ALGORITMOS DE COLORAÇÃO
1425 // Implementações dos algoritmos de coloração de vértices e arestas
1426 // =====
1427
1428 /**
1429  * ALGORITMO DE WELSH-POWELL PARA COLORAÇÃO DE VÉRTICES
1430  *
1431  * Estratégia:
1432  * 1. Ordena os vértices por grau decrescente (mais conexões primeiro)

```

```

1432 * 2. Para cada vértice, atribui a primeira cor disponível
1433 *     que não é usada por nenhum vizinho
1434 *
1435 * Complexidade:  $O(V^2)$  onde  $V$  é o número de vértices
1436 * Garantia: Usa no máximo  $\Delta+1$  cores ( $\Delta$  = grau máximo)
1437 *
1438 * @returns {Object} { cores: {id: numero}, numCores: total }
1439 */
1440 colorirVerticesWelshPowell() {
1441     const vertices = this.obterVertices();
1442     if (!vertices.length) return { cores: {}, numCores: 0 };
1443
1444     // Obtém a lista de adjacência do grafo
1445     const adjacencias = this.obterListaAdjacencia();
1446
1447     // Calcula o grau de cada vértice
1448     const graus = {};
1449     for (const v of vertices) {
1450         graus[v] = (adjacencias[v] || []).length;
1451     }
1452
1453     // Ordena vértices por grau decrescente (estratégia gulosa)
1454     const ordenados = [...vertices].sort((a, b) => graus[b] - graus[a]);
1455     registrarLog(`📋 Welsh-Powell: ordem ${ordenados.map(v => `${v}(g=${graus[v]})`)}
1456     `).join(', ');`);
1457
1458     const cores = {}; // Mapeamento vértice -> cor
1459     let numCores = 0; // Total de cores usadas
1460
1461     // Atribui cor a cada vértice na ordem definida
1462     for (const v of ordenados) {
1463         // Coleta as cores proibidas (já usadas por vizinhos)
1464         const coresProibidas = new Set();
1465         for (const vizinho of (adjacencias[v] || [])) {
1466             if (cores[vizinho] !== undefined) {
1467                 coresProibidas.add(cores[vizinho]);
1468             }
1469         }
1470
1471         // Encontra a primeira cor disponível
1472         let cor = 1;
1473         while (coresProibidas.has(cor)) cor++;
1474
1475         cores[v] = cor;
1476         numCores = Math.max(numCores, cor);
1477
1478         registrarLog(`👤 ${v} (grau ${graus[v]}) → cor ${cor} (proibidas:
1479         ${[...coresProibidas].join(', ') || 'nenhuma'})`);
1480     }
1481
1482     registrarLog(`✅ Welsh-Powell concluído: ${numCores} cores`);
1483     return { cores, numCores };
1484 }
1485
1486 /**
1487 * ALGORITMO ROUND-ROBIN PARA COLORAÇÃO DE ARESTAS DE GRAFOS COMPLETOS ÍMPARES
1488 *
1489 * Funcionamento:
1490 * - Funciona apenas para grafos completos  $K_n$  com  $n$  ímpar
1491 * - Organiza vértices em círculo e um no centro
1492 * - Cada rodada (cor) emparelha vértices opostos
1493 * - O vértice central "descansa" a cada rodada
1494 * - Rotação circular garante que todas as arestas sejam coloridas
1495 *
1496 * Resultado ótimo:  $\chi'(K_n) = n$  para  $n$  ímpar
1497 *
1498 * @param {Object} arestas - Coleção de arestas do Cytoscape
1499 * @param {Array} vertices - Array de IDs dos vértices
1500 * @returns {Object} { cores: {idAresta: cor}, numCores: total }

```

```

1499 */
1500 colorirArestasRoundRobin(arestas, vertices) {
1501     const n = vertices.length; // Número de vértices (n ímpar)
1502     const cores = {}; // Mapeamento aresta -> cor
1503     let circulo = [...vertices]; // Cópia do array para rotação
1504
1505     // Para K_n ímpar, precisamos exatamente de n cores
1506     for (let cor = 1; cor <= n; cor++) {
1507         // Emparelha vértices opostos no círculo
1508         // O vértice na posição 0 fica sem par nesta rodada
1509         for (let i = 0; i < Math.floor(n / 2); i++) {
1510             const v1 = circulo[i + 1]; // Pula o primeiro vértice (centro)
1511             const v2 = circulo[n - 1 - i]; // Vértice oposto
1512
1513             // Encontra a aresta entre v1 e v2
1514             const aresta = arestas.filter(e =>
1515                 (e.data('source') === v1 && e.data('target') === v2) ||
1516                 (e.data('source') === v2 && e.data('target') === v1)
1517             );
1518
1519             if (aresta.length) {
1520                 cores[aresta[0].id()] = cor;
1521             }
1522         }
1523
1524         // Rotação do círculo: primeiro vai para o final
1525         // Isso faz cada vértice alternar como "centro"
1526         circulo.push(circulo.shift());
1527     }
1528
1529     const numCores = new Set(Object.values(cores)).size;
1530     registrarLog(`🌀 Round-Robin para K${n} (ímpar): ${numCores} cores (ótimo)`);
1531     return { cores, numCores };
1532 }
1533
1534 /**
1535  * ALGORITMO GULOSO PARA COLORAÇÃO DE ARESTAS
1536  *
1537  * Estratégia:
1538  * 1. Cria um mapa de incidência entre arestas (quais compartilham vértices)
1539  * 2. Calcula o "grau de incidência" de cada aresta
1540  * 3. Ordena arestas por grau de incidência decrescente
1541  * 4. Atribui a primeira cor disponível para cada aresta
1542  *
1543  * Complexidade:  $O(E^2)$  onde E é o número de arestas
1544  * Garantia: Usa no máximo  $2\Delta-1$  cores (Vizing:  $\Delta+1$  é o ótimo)
1545  *
1546  * @param {Object} arestas - Coleção de arestas do Cytoscape
1547  * @returns {Object} { cores: {idAresta: cor}, numCores: total }
1548  */
1549 colorirArestasGuloso(arestas) {
1550     const listaArestas = arestas.toArray();
1551     if (!listaArestas.length) return { cores: {}, numCores: 0 };
1552
1553     // Constrói mapa de incidência (quais arestas compartilham vértices)
1554     const incidencia = {}; // aresta -> Set de arestas adjacentes
1555     const grauIncidencia = {}; // aresta -> número de arestas adjacentes
1556
1557     for (const a of listaArestas) {
1558         incidencia[a.id()] = new Set();
1559         grauIncidencia[a.id()] = 0;
1560     }
1561
1562     // Calcula adjacência entre todas as arestas ( $O(E^2)$ )
1563     for (let i = 0; i < listaArestas.length; i++) {
1564         const a = listaArestas[i];
1565         const origemA = a.data('source');
1566         const destinoA = a.data('target');

```



```

1568     for (let j = i + 1; j < listaArestas.length; j++) {
1569         const b = listaArestas[j];
1570         const origemB = b.data('source');
1571         const destinoB = b.data('target');
1572
1573         // Duas arestas são adjacentes se compartilham um vértice
1574         if (origemA === origemB || origemA === destinoB ||
1575             destinoA === origemB || destinoA === destinoB) {
1576             incidencia[a.id()].add(b.id());
1577             incidencia[b.id()].add(a.id());
1578             grauIncidencia[a.id()]++;
1579             grauIncidencia[b.id()]++;
1580         }
1581     }
1582 }
1583
1584 // Ordena arestas por grau de incidência (mais restrições primeiro)
1585 const ordenadas = [...listaArestas].sort((a, b) =>
1586     grauIncidencia[b.id()] - grauIncidencia[a.id()]
1587 );
1588
1589 const cores = {};
1590 let numCores = 0;
1591
1592 // Atribui cor a cada aresta
1593 for (const a of ordenadas) {
1594     // Cores proibidas = cores das arestas adjacentes
1595     const proibidas = new Set();
1596     for (const vizinha of incidencia[a.id()]) {
1597         if (cores[vizinha]) proibidas.add(cores[vizinha]);
1598     }
1599
1600     // Encontra primeira cor disponível
1601     let cor = 1;
1602     while (proibidas.has(cor)) cor++;
1603
1604     cores[a.id()] = cor;
1605     numCores = Math.max(numCores, cor);
1606 }
1607
1608 registrarLog(`🤖 Coloração gulosa de arestas: ${numCores} cores`);
1609 return { cores, numCores };
1610 }
1611
1612 /**
1613  * Seleciona o melhor algoritmo para coloração de arestas
1614  * - Usa Round-Robin para grafos completos ímpares
1615  * - Usa Guloso para todos os outros casos
1616  *
1617  * @param {Object} coresVertices - Cores já atribuídas aos vértices
1618  * @returns {Object} Resultado da coloração
1619  */
1620 colorirArestasOtimizado(coresVertices = {}) {
1621     const arestas = this.cy.edges();
1622     if (!arestas.length) return { cores: {}, numCores: 0 };
1623
1624     const vertices = this.obterVertices();
1625     const n = vertices.length;
1626
1627     // Verifica se é um grafo completo não orientado
1628     const totalPossivel = this.estado.orientado
1629         ? n * (n - 1) // Fórmula para grafo orientado
1630         : (n * (n - 1)) / 2; // Fórmula para não orientado
1631
1632     const ehCompleto = arestas.length === totalPossivel && !this.estado.orientado;
1633
1634     // Para K_n ímpar, Round-Robin dá o resultado ótimo
1635     if (ehCompleto && n % 2 === 1) {
1636         registrarLog(`🐞 Grafo completo K${n} (ímpar) → usando Round-Robin`);

```



```

1637     return this.colorirArestasRoundRobin(arestas, vertices);
1638 }
1639
1640 // Para todos os outros casos, usa o algoritmo guloso
1641 return this.colorirArestasGuloso(arestas);
1642 }
1643
1644 /**
1645  * Aplica visualmente as cores calculadas aos vértices
1646  * @param {Object} cores - Mapeamento {idVertice: numeroCor}
1647  */
1648 aplicarCoresVertices(cores) {
1649     for (const [id, indiceCor] of Object.entries(cores)) {
1650         const cor = PALETA_VERTICES[(indiceCor - 1) % PALETA_VERTICES.length];
1651         const no = this.cy.getElementById(id);
1652         if (no.length) {
1653             no.style('background-color', cor);
1654             no.style('border-color', '#2C3E50');
1655             no.style('color', obterCorContraste(cor)); // Texto legível
1656         }
1657     }
1658     registrarLog(`🎨 ${Object.keys(cores).length} vértices coloridos com ${new Set(Object.values(cores)).size} cores.`);
1659 }
1660
1661 /**
1662  * Aplica visualmente as cores calculadas às arestas
1663  * @param {Object} cores - Mapeamento {idAresta: numeroCor}
1664  */
1665 aplicarCoresArestas(cores) {
1666     if (!cores || !Object.keys(cores).length) {
1667         registrarLog('⚠️ Nenhuma cor para arestas.');
```

1668 return;

1669 }

1670

```

1671     for (const [id, indiceCor] of Object.entries(cores)) {
1672         const cor = PALETA_arestas[(indiceCor - 1) % PALETA_arestas.length];
1673         const aresta = this.cy.getElementById(id);
1674         if (aresta.length) {
1675             aresta.style('line-color', cor);
1676             aresta.style('target-arrow-color', cor);
1677             aresta.style('width', 3); // Mais grossa para destacar
1678         }
1679     }
1680 }
1681
1682 /** Restaura as cores padrão de todos os elementos */
1683 resetarCores() {
1684     this.cy.nodes().forEach(n => {
1685         n.style('background-color', '#a5d8ff');
1686         n.style('border-color', '#1971c2');
1687         n.style('color', '#1e293b');
1688     });
1689
1690     this.cy.edges().forEach(e => {
1691         e.style('line-color', '#868e96');
1692         e.style('target-arrow-color', '#868e96');
1693         e.style('width', 2);
1694     });
1695
1696     registrarLog('🔄 Cores resetadas para o padrão.');
```

1697 }

1698

1699 /**

1700 * Executa a coloração completa do grafo (vértices e arestas)

1701 * e exibe estatísticas comparando com limites teóricos

1702 */

```

1703 colorirGrafoCompleto() {
1704     if (!this.cy.nodes().length) {
```

```

1705     exibirNotificacao('Crie vértices primeiro', NOTIFICACAO.AVISO);
1706     return;
1707 }
1708
1709 registrarLog('— COLORAÇÃO COMPLETA —');
1710 registrarLog('⚡ Vértices (Welsh-Powell):');
1711
1712 const resultadoVertices = this.colorirVerticesWelshPowell();
1713 this.aplicarCoresVertices(resultadoVertices.cores);
1714
1715 if (this.cy.edges().length) {
1716     registrarLog('⚡ Arestas:');
1717     const resultadoArestas = this.colorirArestasOtimizado(resultadoVertices.cores);
1718     this.aplicarCoresArestas(resultadoArestas.cores);
1719
1720     const vertices = this.obterVertices();
1721     const adjacencias = this.obterListaAdjacencia();
1722     const grauMaximo = Math.max(...vertices.map(v => (adjacencias[v] || []).length));
1723
1724     registrarLog(`📊 Resumo: V=${vertices.length} (${resultadoVertices.numCores} cores) | E=${this.cy.edges().length} (${resultadoArestas.numCores} cores) | Δ=${grauMaximo}`);
1725
1726     // Compara com limites teóricos (Teorema de Vizing)
1727     if (resultadoArestas.numCores === grauMaximo) {
1728         registrarLog(`✅ Ótimo (χ=Δ=${grauMaximo})`);
1729     } else if (resultadoArestas.numCores === grauMaximo + 1) {
1730         registrarLog(`✅ Dentro de Vizing (Δ+1=${grauMaximo + 1})`);
1731     }
1732
1733     exibirNotificacao(`Coloração: ${resultadoVertices.numCores} cores vértices, ${resultadoArestas.numCores} cores arestas`, NOTIFICACAO.SUCESSO);
1734 } else {
1735     registrarLog(`✅ Apenas vértices: ${resultadoVertices.numCores} cores`);
1736     exibirNotificacao(`Vértices coloridos com ${resultadoVertices.numCores} cores`, NOTIFICACAO.SUCESSO);
1737 }
1738 }
1739
1740 // =====
1741 // ANIMAÇÃO DA COLORAÇÃO PASSO A PASSO
1742 // =====
1743
1744 /**
1745  * Executa a coloração com animação, mostrando cada passo
1746  * Permite interrupção a qualquer momento
1747  */
1748 async executarColoracaoAnimada() {
1749     if (this.estado.executandoAnimacao) {
1750         registrarLog('⏸️ Aguarde a animação atual terminar...');
1751         return;
1752     }
1753
1754     if (!this.cy.nodes().length) {
1755         exibirNotificacao('Crie vértices primeiro', NOTIFICACAO.AVISO);
1756         return;
1757     }
1758
1759     this.definirEstadoExecucao(true);
1760     this.limparCoresBusca();
1761
1762     const idExecucao = ++this.estado.idExecucaoAtual;
1763     registrarLog('— ANIMAÇÃO DA COLORAÇÃO TOTAL —');
1764
1765     try {
1766         // FASE 1: Coloração dos vértices
1767         const adjacencias = this.obterListaAdjacencia();
1768         const vertices = this.obterVertices();
1769         const graus = {};

```

```

1770   for (const v of vertices) {
1771       graus[v] = (adjacencias[v] || []).length;
1772   }
1773
1774   const ordenados = [...vertices].sort((a, b) => graus[b] - graus[a]);
1775   registrarLog(`🌀 FASE 1: Vértices (ordem: ${ordenados.map(v => `${v}(g=${graus[v]
1776   ]})`).join(', ')}`);
1777
1778   const coresVertices = {};
1779
1780   for (const v of ordenados) {
1781       // Permite cancelamento
1782       if (idExecucao !== this.estado.idExecucaoAtual) {
1783           registrarLog('🛑 Animação interrompida');
1784           this.definirEstadoExecucao(false);
1785           return;
1786       }
1787
1788       const proibidas = new Set();
1789       for (const vizinho of (adjacencias[v] || [])) {
1790           if (coresVertices[vizinho.para]) proibidas.add(coresVertices[vizinho.para]);
1791       }
1792
1793       let cor = 1;
1794       while (proibidas.has(cor)) cor++;
1795       coresVertices[v] = cor;
1796
1797       const corHexadecimal = PALETA_VERTICES[(cor - 1) % PALETA_VERTICES.length];
1798       this.cy.getElementById(v)
1799           .style('background-color', corHexadecimal)
1800           .style('border-color', '#2C3E50')
1801           .style('color', obterCorContraste(corHexadecimal));
1802
1803       registrarLog(`👤 ${v} → cor ${cor}`);
1804       this.centralizarNoVerticeSeAtivado(v);
1805       await this.aguardar(this.estado.velocidadeAnimacao);
1806   }
1807
1808   const numCoresVertices = new Set(Object.values(coresVertices)).size;
1809   registrarLog(`✅ Vértices coloridos com ${numCoresVertices} cores`);
1810
1811   // FASE 2: Coloração das arestas
1812   if (this.cy.edges().length) {
1813       registrarLog('🌀 FASE 2: Arestas');
1814
1815       const listaVertices = this.obterVertices();
1816       const totalPossivel = this.estado.orientado
1817           ? listaVertices.length * (listaVertices.length - 1)
1818           : (listaVertices.length * (listaVertices.length - 1)) / 2;
1819
1820       const ehCompleto = this.cy.edges().length === totalPossivel && !
1821       this.estado.orientado;
1822
1823       if (ehCompleto && listaVertices.length % 2 === 1) {
1824           // Grafo completo ímpar: usa Round-Robin (resultado instantâneo)
1825           registrarLog(`👤 Grafo completo K${listaVertices.length} (ímpar) → usando
1826           Round-Robin`);
1827           const coresArestas = this.colorirArestasRoundRobin(this.cy.edges(),
1828           listaVertices);
1829           this.aplicarCoresArestas(coresArestas.cores);
1830
1831           const grauMaximo = Math.max(...listaVertices.map(v => graus[v]));
1832           registrarLog(`📊 Resumo: V=${listaVertices.length} (${numCoresVertices}
1833           cores) | E=${this.cy.edges().length} (${coresArestas.numCores} cores) |
1834           Δ=${grauMaximo}`);
1835       } else {
1836           // Outros grafos: animação do algoritmo guloso passo a passo
1837           registrarLog('👤 Aplicando algoritmo guloso para arestas');
1838       }
1839   }

```

```

const listaArestas = this.cy.edges().toArray();

// Constrói mapa de incidência
const inc = {};
const grauInc = {};

for (const a of listaArestas) {
  inc[a.id()] = new Set();
  grauInc[a.id()] = 0;
}

for (let i = 0; i < listaArestas.length; i++) {
  const a = listaArestas[i];
  const origemA = a.data('source');
  const destinoA = a.data('target');

  for (let j = i + 1; j < listaArestas.length; j++) {
    const b = listaArestas[j];
    const origemB = b.data('source');
    const destinoB = b.data('target');

    if (origemA === origemB || origemA === destinoB ||
        destinoA === origemB || destinoA === destinoB) {
      inc[a.id()].add(b.id());
      inc[b.id()].add(a.id());
      grauInc[a.id()]++;
      grauInc[b.id()]++;
    }
  }
}

const ordenadas = [...listaArestas].sort((a, b) => grauInc[b.id()] - grauInc[a.id()]);
const coresArestas = {};

for (const a of ordenadas) {
  if (idExecucao !== this.estado.idExecucaoAtual) {
    registrarLog('⊖ Animação interrompida');
    this.definirEstadoExecucao(false);
    return;
  }

  const proibidas = new Set();
  for (const vizinha of inc[a.id()]) {
    if (coresArestas[vizinha]) proibidas.add(coresArestas[vizinha]);
  }

  let cor = 1;
  while (proibidas.has(cor)) cor++;
  coresArestas[a.id()] = cor;

  const corHexadecimal = PALETA_ARESTAS[(cor - 1) % PALETA_ARESTAS.length];
  a.style('line-color', corHexadecimal)
    .style('target-arrow-color', corHexadecimal)
    .style('width', 3);

  registrarLog(`👁 Aresta ${a.data('source')}→${a.data('target')} → cor ${cor}`);
  await this.aguardar(this.estado.velocidadeAnimacao);
}

const numCoresArestas = new Set(Object.values(coresArestas)).size;
registrarLog(`✅ Arestas coloridas com ${numCoresArestas} cores`);

const grauMaximo = Math.max(...listaVertices.map(v => graus[v]));
registrarLog(`📊 Resumo: V=${listaVertices.length} (${numCoresVertices} cores) | E=${this.cy.edges().length} (${numCoresArestas} cores) | Δ=${grauMaximo}`);

```

```

1898     }
1899 }
1900
1901     registrarLog('🎉 Animação finalizada com sucesso!');
1902     exibirNotificacao('Animação de coloração concluída!', NOTIFICACAO.SUCESSO);
1903 } catch (erro) {
1904     registrarLog(`❌ Erro na animação: ${erro.message}`);
1905     exibirNotificacao('Erro durante a animação', NOTIFICACAO.ERRO);
1906 } finally {
1907     this.definirEstadoExecucao(false);
1908 }
1909 }
1910
1911 // =====
1912 // OPERAÇÕES DE ARQUIVO (EXPORTAR/IMPORTAR)
1913 // =====
1914
1915 /**
1916  * Exporta o grafo para formato texto
1917  * Formato:
1918  *   Linha 1: V E (número de vértices e arestas)
1919  *   Linha 2: orientado ponderado (0/1 para cada)
1920  *   Linhas seguintes: arestas (origem destino [peso])
1921  *   --- (separador)
1922  *   Linhas finais: posições dos vértices (id x y)
1923  *
1924  * @returns {string} Conteúdo do arquivo
1925  */
1926 exportarParaTexto() {
1927     const linhas = [];
1928     const numVertices = this.cy.nodes().length;
1929     const numArestas = this.cy.edges().length;
1930
1931     // Cabeçalho
1932     linhas.push(`${numVertices} ${numArestas}`);
1933     linhas.push(`${this.estado.orientado ? 1 : 0} ${this.estado.ponderado ? 1 : 0}`);
1934
1935     // Lista de arestas
1936     this.cy.edges().forEach(aresta => {
1937         const origem = aresta.data('source');
1938         const destino = aresta.data('target');
1939         const peso = aresta.data('label');
1940
1941         if (this.estado.ponderado && peso && peso !== '') {
1942             linhas.push(`${origem} ${destino} ${peso}`);
1943         } else {
1944             linhas.push(`${origem} ${destino}`);
1945         }
1946     });
1947
1948     // Separador
1949     linhas.push('---');
1950
1951     // Posições dos vértices
1952     this.cy.nodes().forEach(no => {
1953         const pos = no.position();
1954         linhas.push(`${no.id()} ${Math.round(pos.x)} ${Math.round(pos.y)}`);
1955     });
1956
1957     return linhas.join('\n');
1958 }
1959
1960 /**
1961  * Importa um grafo a partir de texto no formato .txt
1962  * @param {string} texto - Conteúdo do arquivo
1963  */
1964 importarDeTexto(texto) {
1965     if (this.estado.executandoAnimacao) {
1966         registrarLog('🚫 Operação bloqueada durante animação');

```

```

1967     return;
1968 }
1969
1970 try {
1971     const linhas = texto.split(/\r?\n/).map(l => l.trim()).filter(l => l !== '');
1972
1973     if (linhas.length < 2) {
1974         throw new Error('Arquivo muito curto');
1975     }
1976
1977     // Lê cabeçalho
1978     const [numV, numE] = linhas[0].split(/\s+/).map(Number);
1979     const metadados = linhas[1].split(/\s+/);
1980
1981     if (metadados.length < 2) {
1982         throw new Error('Formato inválido na segunda linha');
1983     }
1984
1985     const orientado = metadados[0] === '1';
1986     const ponderado = metadados[1] === '1';
1987
1988     // Limpa grafo atual
1989     this.cy.elements().remove();
1990     this.estado.contadorVertices = 1;
1991     this.estado.orientado = orientado;
1992     this.estado.ponderado = ponderado;
1993
1994     // Atualiza interface
1995     if (pegarElemento('selTipo')) pegarElemento('selTipo').value = orientado ?
        'orientado' : 'naorientado';
1996     if (pegarElemento('selPeso')) pegarElemento('selPeso').value = ponderado ? 'sim' :
        'nao';
1997
1998     let modoPosicoes = false;
1999     const posicoes = {};
2000
2001     // Processa Linhas
2002     for (let i = 2; i < linhas.length; i++) {
2003         const linha = linhas[i];
2004
2005         if (linha === '---') {
2006             modoPosicoes = true; // Mudou para seção de posições
2007             continue;
2008         }
2009
2010         const partes = linha.split(/\s+/);
2011
2012         if (!modoPosicoes) {
2013             // Seção de arestas
2014             if (partes.length < 2) continue;
2015
2016             const origem = partes[0];
2017             const destino = partes[1];
2018             const peso = (partes.length >= 3 && ponderado) ? partes[2] : null;
2019
2020             // Cria vértices se não existirem
2021             if (!this.cy.getElementById(origem).length) this.adicionarVertice(origem);
2022             if (!this.cy.getElementById(destino).length) this.adicionarVertice(destino);
2023             this.adicionarAresta(origem, destino, peso, false);
2024         } else if (partes.length >= 3) {
2025             // Seção de posições
2026             posicoes[partes[0]] = {
2027                 x: parseFloat(partes[1]),
2028                 y: parseFloat(partes[2])
2029             };
2030         }
2031     }
2032
2033     // Aplica posições salvas

```

```

2034     for (const [id, pos] of Object.entries(posicoes)) {
2035         const no = this.cy.getElementById(id);
2036         if (no.length) no.position(pos);
2037     }
2038
2039     this.atualizarEstiloArestas();
2040     this.atualizarIndicadores();
2041     this.cy.fit(undefined, 50); // Ajusta zoom
2042
2043     registrarLog(`📁 Arquivo carregado: ${this.cy.nodes().length} vértices,
2044     ${this.cy.edges().length} arestas.`);
2045     exibirNotificacao(`Grafo carregado: ${this.cy.nodes().length}V,
2046     ${this.cy.edges().length}E`, NOTIFICACAO.SUCESSO);
2047 } catch (erro) {
2048     registrarLog(`❌ Erro ao importar: ${erro.message}`);
2049     exibirNotificacao('Erro ao carregar arquivo', NOTIFICACAO.ERRO);
2050 }
2051
2052 // =====
2053 // CARREGAMENTO DE MODELOS PRÉ-DEFINIDOS
2054 // =====
2055
2056 /**
2057  * Carrega um modelo de grafo pré-definido
2058  * @param {string} tipo - Identificador do modelo
2059  */
2060 carregarModelo(tipo) {
2061     if (this.estado.executandoAnimacao) {
2062         registrarLog('🛑 Operação bloqueada durante animação');
2063         return;
2064     }
2065
2066     this.cy.elements().remove();
2067     this.estado.contadorVertices = 1;
2068     this.limparCoresBusca();
2069
2070     // Funções auxiliares para criar vértices e arestas
2071     const adicionarVerticeModelo = (id, x, y) => {
2072         this.cy.add({ group: 'nodes', data: { id } }).position({ x, y });
2073     };
2074
2075     const adicionarArestaModelo = (origem, destino, peso = '') => {
2076         this.cy.add({
2077             group: 'edges',
2078             data: {
2079                 id: `e_${origem}_${destino}_${Date.now()}`,
2080                 source: origem,
2081                 target: destino,
2082                 label: peso
2083             }
2084         });
2085     };
2086
2087     // Definição dos modelos disponíveis
2088     const modelos = {
2089         // K5: Grafo completo com 5 vértices (todos conectados entre si)
2090         'completo5': () => {
2091             const ids = ['A', 'B', 'C', 'D', 'E'];
2092             // Posiciona vértices em círculo
2093             ids.forEach((id, i) => {
2094                 const angulo = (i * 2 * Math.PI / 5) - Math.PI / 2;
2095                 adicionarVerticeModelo(id, 400 + 150 * Math.cos(angulo), 300 + 150 * Math.sin(
2096                     angulo));
2097             });
2098             // Conecta todos com todos
2099             for (let i = 0; i < 5; i++) {
2100                 for (let j = i + 1; j < 5; j++) {
2101                     adicionarArestaModelo(ids[i], ids[j]);
2102                 }
2103             }
2104         }
2105     };

```



```

2100     }
2101   }
2102   registrarLog('📦 K5 (Grafo completo com 5 vértices) carregado.');
```

2103 },

2104

2105 *// P5: Caminho simples com 5 vértices em linha*

2106 'caminho5': () => {

2107 ['A', 'B', 'C', 'D', 'E'].forEach((id, i) => adicionarVerticeModelo(id, 200 + i * 100, 300));

2108 for (let i = 0; i < 4; i++) {

2109 adicionarArestaModelo(String.fromCharCode(65 + i), String.fromCharCode(66 + i));

2110 }

2111 registrarLog('📦 P5 (Caminho com 5 vértices) carregado.');

2112 },

2113

2114 *// C5: Ciclo com 5 vértices*

2115 'ciclo5': () => {

2116 ['A', 'B', 'C', 'D', 'E'].forEach((id, i) => {

2117 const angulo = (i * 2 * Math.PI / 5) - Math.PI / 2;

2118 adicionarVerticeModelo(id, 400 + 150 * Math.cos(angulo), 300 + 150 * Math.sin(angulo));

2119 });

2120 for (let i = 0; i < 4; i++) {

2121 adicionarArestaModelo(String.fromCharCode(65 + i), String.fromCharCode(66 + i));

2122 }

2123 adicionarArestaModelo('E', 'A'); *// Fecha o ciclo*

2124 registrarLog('📦 C5 (Ciclo com 5 vértices) carregado.');

2125 },

2126

2127 *// K3,3: Grafo bipartido completo (dois conjuntos de 3)*

2128 'bipartido': () => {

2129 ['A1', 'A2', 'A3'].forEach((id, i) => adicionarVerticeModelo(id, 250, 200 + i * 80));

2130 ['B1', 'B2', 'B3'].forEach((id, i) => adicionarVerticeModelo(id, 450, 200 + i * 80));

2131 for (const e of ['A1', 'A2', 'A3']) {

2132 for (const d of ['B1', 'B2', 'B3']) {

2133 adicionarArestaModelo(e, d);

2134 }

2135 }

2136 registrarLog('📦 K3,3 (Grafo bipartido completo) carregado.');

2137 },

2138

2139 *// Quadrado com diagonal (grafo simples para testes)*

2140 'simples': () => {

2141 adicionarVerticeModelo('A', 250, 200);

2142 adicionarVerticeModelo('B', 450, 200);

2143 adicionarVerticeModelo('C', 450, 400);

2144 adicionarVerticeModelo('D', 250, 400);

2145 adicionarArestaModelo('A', 'B');

2146 adicionarArestaModelo('B', 'C');

2147 adicionarArestaModelo('C', 'D');

2148 adicionarArestaModelo('D', 'A');

2149 adicionarArestaModelo('A', 'C'); *// Diagonal*

2150 registrarLog('📦 Grafo simples (quadrado com diagonal) carregado.');

2151 },

2152

2153 *// Margaridas: grafo orientado complexo com 20 vértices*

2154 'margaridas': () => {

2155 this.estado.orientado = true;

2156 if (pegarElemento('selTipo')) pegarElemento('selTipo').value = 'orientado';

2157

2158 const posicoes = {

2159 'V1': [200, 300], 'V2': [100, 155], 'V3': [140, 455],

2160 'V4': [30, 310], 'V5': [315, 200], 'V6': [45, 220],

2161 'V7': [250, 150], 'V8': [50, 400], 'V9': [250, 450],

2162 'V10': [175, 135], 'V11': [600, 300], 'V12': [500, 200],


```

2163         'V13': [565, 150], 'V14': [655, 155], 'V15': [730, 205],
2164         'V16': [755, 300], 'V17': [725, 400], 'V18': [655, 450],
2165         'V19': [560, 450], 'V20': [500, 400]
2166     };
2167
2168     for (const [id, [x, y]] of Object.entries(posicoes)) {
2169         adicionarVerticeModelo(id, x, y);
2170     }
2171
2172     const arestas = [
2173         ['V1', 'V2'], ['V2', 'V1'], ['V1', 'V3'], ['V3', 'V1'],
2174         ['V1', 'V4'], ['V4', 'V1'], ['V1', 'V5'], ['V5', 'V1'],
2175         ['V1', 'V6'], ['V6', 'V1'], ['V1', 'V7'], ['V7', 'V1'],
2176         ['V1', 'V8'], ['V8', 'V1'], ['V1', 'V9'], ['V9', 'V1'],
2177         ['V1', 'V10'], ['V10', 'V1'], ['V1', 'V11'], ['V11', 'V1'],
2178         ['V11', 'V12'], ['V12', 'V11'], ['V11', 'V13'], ['V13', 'V11'],
2179         ['V11', 'V14'], ['V14', 'V11'], ['V11', 'V15'], ['V15', 'V11'],
2180         ['V11', 'V16'], ['V16', 'V11'], ['V11', 'V17'], ['V17', 'V11'],
2181         ['V11', 'V18'], ['V18', 'V11'], ['V11', 'V19'], ['V19', 'V11'],
2182         ['V11', 'V20'], ['V20', 'V11']
2183     ];
2184
2185     arestas.forEach(([u, v]) => adicionarArestaModelo(u, v));
2186     registrarLog('👑 Margaridas (20 vértices, 40 arestas) carregado.');
```

```

2187     }
2188 };
2189
2190 if (modelos[tipo]) {
2191     modelos[tipo]();
2192     this.atualizarEstiloArestas();
2193     this.atualizarIndicadores();
2194     this.cy.fit(undefined, 50);
2195     registrarLog(`👑 Modelo "${tipo}" carregado!`);
2196     exibirNotificacao(`Modelo "${tipo}" carregado`, NOTIFICACAO.SUCESSO);
2197 } else if (tipo !== 'vazio') {
2198     exibirNotificacao(`Modelo "${tipo}" não encontrado`, NOTIFICACAO.ERRO);
2199 }
2200 }
2201
2202 // =====
2203 // MÉTODOS AUXILIARES
2204 // =====
2205
2206 /** @returns {Array} Lista de IDs de todos os vértices */
2207 obterVertices() {
2208     return this.cy.nodes().map(n => n.id());
2209 }
2210
2211 /**
2212  * Constrói lista de adjacência do grafo
2213  * Para grafos não orientados, adiciona arestas em ambas as direções
2214  * @returns {Object} {vertice: [{para: destino, peso: valor}]}
2215  */
2216 obterListaAdjacencia() {
2217     const adjacencias = {};
2218     this.cy.nodes().forEach(no => {
2219         adjacencias[no.id()] = [];
2220     });
2221
2222     this.cy.edges().forEach(e => {
2223         const origem = e.data('source');
2224         const destino = e.data('target');
2225         const peso = e.data('label') ? Number(e.data('label')) : null;
2226
2227         adjacencias[origem].push({ para: destino, peso: peso });
2228         // Para grafos não orientados, adiciona a aresta reversa
2229         if (!this.estado.orientado && origem !== destino) {
2230             adjacencias[destino].push({ para: origem, peso: peso });
2231         }
2232     });

```

```

2232     });
2233
2234     return adjacencias;
2235 }
2236
2237 /** Atualiza todos os indicadores estatísticos na interface */
2238 atualizarIndicadores() {
2239     const numVertices = this.cy.nodes().length;
2240     const numArestas = this.cy.edges().length;
2241
2242     // Atualiza badges e labels
2243     if (pegarElemento('badgeStats')) pegarElemento('badgeStats').textContent =
2244         `V:${numVertices} E:${numArestas}`;
2245     if (pegarElemento('badgeStatsTop')) pegarElemento('badgeStatsTop').textContent =
2246         `V:${numVertices} E:${numArestas}`;
2247     if (pegarElemento('infoVertices')) pegarElemento('infoVertices').textContent =
2248         numVertices;
2249     if (pegarElemento('infoArestas')) pegarElemento('infoArestas').textContent =
2250         numArestas;
2251
2252     // Calcula densidade e grau médio
2253     if (numVertices > 1) {
2254         const densidade = this.estado.orientado
2255             ? numArestas / (numVertices * (numVertices - 1))
2256             : (2 * numArestas) / (numVertices * (numVertices - 1));
2257
2258         if (pegarElemento('infoDensidade')) {
2259             pegarElemento('infoDensidade').textContent = densidade.toFixed(4);
2260         }
2261         if (pegarElemento('infoGrauMedio')) {
2262             pegarElemento('infoGrauMedio').textContent = (2 * numArestas / numVertices).
2263                 toFixed(2);
2264         }
2265     } else {
2266         if (pegarElemento('infoDensidade')) pegarElemento('infoDensidade').textContent =
2267             '0.00';
2268         if (pegarElemento('infoGrauMedio')) pegarElemento('infoGrauMedio').textContent =
2269             '0.00';
2270     }
2271
2272     this.atualizarComponentesConexas();
2273 }
2274
2275 /** Calcula o número de componentes conexas usando DFS iterativa */
2276 atualizarComponentesConexas() {
2277     const visitados = new Set();
2278     let componentes = 0;
2279     const adjacencias = this.obterListaAdjacencia();
2280
2281     for (const v of this.obterVertices()) {
2282         if (!visitados.has(v)) {
2283             componentes++;
2284             const pilha = [v];
2285             visitados.add(v);
2286
2287             // DFS iterativa (evita estouro de pilha em grafos grandes)
2288             while (pilha.length) {
2289                 const atual = pilha.pop();
2290                 for (const { para } of (adjacencias[atual] || [])) {
2291                     if (!visitados.has(para)) {
2292                         visitados.add(para);
2293                         pilha.push(para);
2294                     }
2295                 }
2296             }
2297         }
2298     }
2299 }
2300
2301 if (pegarElemento('infoComponentes')) {

```

```

2294     pegarElemento('infoComponentes').textContent = componentes;
2295 }
2296 }
2297
2298 /** Atualiza o estilo visual das arestas conforme configurações */
2299 atualizarEstiloArestas() {
2300     this.cy.edges().forEach(e => {
2301         e.style('curve-style', this.estado.arestasCurvas ? 'bezier' : 'haystack');
2302
2303         if (this.estado.arestasCurvas) {
2304             e.style('control-point-step-size', 40);
2305         }
2306
2307         e.style('target-arrow-shape', this.estado.orientado ? 'triangle' : 'none');
2308
2309         const temRotulo = this.estado.ponderado && e.data('label') && e.data('label') !==
            '';
2310         if (temRotulo) {
2311             e.addClass('show-label');
2312         } else {
2313             e.removeClass('show-label');
2314         }
2315
2316         e.style('label', temRotulo ? e.data('label') : '');
2317     });
2318 }
2319
2320 /** Habilita/desabilita botões durante animação */
2321 definirEstadoExecucao(executando) {
2322     this.estado.executandoAnimacao = executando;
2323
2324     const botoes = [
2325         'modeAddVertex', 'modeAddEdge', 'modeSelect',
2326         'btnAddV', 'btnRemV', 'btnAddE', 'btnRemE',
2327         'btnBatchAdd', 'btnAutoEdges', 'btnAutoEdgesClear',
2328         'btnClear', 'btnStabilize', 'btnUndo', 'btnRedo',
2329         'btnColorirVertices', 'btnColorirArestas', 'btnColorirTudo',
2330         'btnColorirAnimado', 'btnColorirReset',
2331         'btnCarregarModelo', 'btnSaveFile', 'btnLoadFile'
2332     ];
2333
2334     botoes.forEach(id => {
2335         const elemento = pegarElemento(id);
2336         if (elemento) elemento.disabled = executando;
2337     });
2338 }
2339
2340 /** Aguarda um tempo em milissegundos (para animações) */
2341 aguardar(ms) {
2342     return new Promise(resolve => setTimeout(resolve, ms));
2343 }
2344
2345 /** Limpa cores de destaque e incrementa ID de execução (cancela animações) */
2346 limparCoresBusca() {
2347     this.estado.idExecucaoAtual++;
2348
2349     this.cy.nodes().forEach(n => {
2350         n.style('background-color', '#a5d8ff');
2351         n.style('border-color', '#1971c2');
2352     });
2353
2354     this.cy.edges().forEach(e => {
2355         e.style('line-color', '#868e96');
2356         e.style('target-arrow-color', '#868e96');
2357         e.style('width', 2);
2358     });
2359 }
2360
2361 /** Centraliza a visualização em um vértice específico (se opção ativada) */

```

```

2362     centralizarNoVerticeSeAtivado(idVertice) {
2363         if (pegarElemento('swAutoFocus')?.checked) {
2364             this.cy.animate({
2365                 fit: { eles: this.cy.getElementById(idVertice), padding: 100 },
2366                 duration: 280
2367             });
2368         }
2369     }
2370
2371     /** Define o modo de operação atual */
2372     definirModo(modo) {
2373         if (this.estado.executandoAnimacao) return;
2374
2375         this.estado.modo = modo;
2376         this.estado.arestaOrigem = null;
2377
2378         const botaoVertice = pegarElemento('modeAddVertex');
2379         const botaoAresta = pegarElemento('modeAddEdge');
2380         const botaoSelecionar = pegarElemento('modeSelect');
2381
2382         if (botaoVertice) botaoVertice.classList.toggle('mode-active', modo === MODOS.VERTICE);
2383         if (botaoAresta) botaoAresta.classList.toggle('mode-active', modo === MODOS.ARESTA);
2384         if (botaoSelecionar) botaoSelecionar.classList.toggle('mode-active', modo === MODOS.SELECIONAR);
2385
2386         const nomeModo = {
2387             [MODOS.VERTICE]: 'Inserir vértice',
2388             [MODOS.ARESTA]: 'Inserir aresta',
2389             [MODOS.SELECIONAR]: 'Selecionar'
2390         };
2391
2392         registrarLog(`Modo: ${nomeModo[modo] || modo}`);
2393     }
2394
2395     /** Gera o próximo ID automático para vértices (V1, V2, V3...) */
2396     proximoIdVertice() {
2397         while (this.cy.getElementById(`V${this.estado.contadorVertices}`).length) {
2398             this.estado.contadorVertices++;
2399         }
2400         return `V${this.estado.contadorVertices++}`;
2401     }
2402
2403     /** Abre diálogo para editar peso de uma aresta */
2404     editarPesoAresta(idAresta) {
2405         if (!this.estado.ponderado) {
2406             exibirNotificacao('Ative "Com peso" primeiro', NOTIFICACAO.AVISO);
2407             return;
2408         }
2409
2410         const aresta = this.cy.getElementById(idAresta);
2411         if (!aresta.length) return;
2412
2413         const pesoAtual = aresta.data('label') || '';
2414         const novoPeso = prompt(`Novo peso para ${aresta.data('source')}→${aresta.data('target')}:`, pesoAtual);
2415
2416         if (novoPeso !== null) {
2417             const rotuloAntigo = aresta.data('label');
2418
2419             if (novoPeso === '' || isNaN(Number(novoPeso))) {
2420                 aresta.data('label', '');
2421             } else {
2422                 aresta.data('label', String(novoPeso));
2423             }
2424
2425             this.historico.adicionar({
2426                 tipo: 'editar-peso-aresta',
2427                 dados: { id: idAresta, rotuloAntigo, rotuloNovo: aresta.data('label') },

```

```

2428     desfazer: () => {
2429         const edge = this.cy.getElementById(idAresta);
2430         if (edge.length) {
2431             edge.data('label', rotuloAntigo);
2432             this.atualizarEstiloArestas();
2433         }
2434     },
2435     refazer: () => {
2436         const edge = this.cy.getElementById(idAresta);
2437         if (edge.length) {
2438             edge.data('label', aresta.data('label'));
2439             this.atualizarEstiloArestas();
2440         }
2441     }
2442 });
2443
2444 this.atualizarEstiloArestas();
2445 registrarLog(`🔄 Peso alterado para ${novoPeso || 'removido'} em
2446   ${aresta.data('source')}->${aresta.data('target')}}`);
2447 }
2448 }
2449
2450 // =====
2451 // CONEXÃO DE EVENTOS
2452 // =====
2453
2454 /** Conecta eventos do Cytoscape (cliques, seleções, menu contexto) */
2455 conectarEventosCytoscape() {
2456     // Clique no grafo
2457     this.cy.on('tap', (e) => {
2458         if (this.estado.executandoAnimacao) return;
2459         const alvo = e.target;
2460
2461         // Modo vértice: clique na área vazia cria vértice
2462         if (this.estado.modo === MODOS.VERTECE && alvo === this.cy) {
2463             this.adicionarVertice(this.proximoIdVertice(), e.position);
2464         }
2465         // Modo aresta: clicar em vértices cria aresta
2466         else if (this.estado.modo === MODOS.ARESTA && alvo.isNode()) {
2467             if (!this.estado.arestaOrigem) {
2468                 // Primeiro vértice: define origem
2469                 this.estado.arestaOrigem = alvo.id();
2470                 alvo.select();
2471                 alvo.addClass('pendente');
2472                 registrarLog(`📍 Origem: ${this.estado.arestaOrigem}. Clique no destino.`);
2473             } else {
2474                 // Segundo vértice: cria aresta
2475                 this.adicionarAresta(this.estado.arestaOrigem, alvo.id());
2476                 const origem = this.cy.getElementById(this.estado.arestaOrigem);
2477                 if (origem.length) origem.removeClass('pendente');
2478                 this.estado.arestaOrigem = null;
2479                 this.cy.elements().unselect();
2480             }
2481         }
2482     });
2483
2484     // Duplo clique em aresta: editar peso
2485     this.cy.on('dbltap', 'edge', (e) => {
2486         if (this.estado.ponderado) {
2487             this.editarPesoAresta(e.target.id());
2488         }
2489     });
2490
2491     // Eventos de seleção
2492     this.cy.on('select', 'node', (e) => {
2493         this.estado.verticeSelecionadoId = e.target.id();
2494         this.estado.arestaSelecionadaId = null;
2495     });

```

```

2496     this.cy.on('select', 'edge', (e) => {
2497         this.estado.arestaSelecionadaId = e.target.id();
2498         this.estado.verticeSelecionadoId = null;
2499     });
2500
2501     this.cy.on('unselect', () => {
2502         this.estado.verticeSelecionadoId = null;
2503         this.estado.arestaSelecionadaId = null;
2504     });
2505
2506     // Menu de contexto (clique direito)
2507     this.cy.on('cxttap', (e) => {
2508         if (this.estado.executandoAnimacao) return;
2509
2510         const alvo = e.target;
2511         this.estado.ctxVerticeId = alvo.isNode() ? alvo.id() : null;
2512         this.estado.ctxArestaId = alvo.isEdge() ? alvo.id() : null;
2513         this.estado.ctxPosicaoTela = e.position;
2514
2515         const menu = pegarElemento('ctxMenu');
2516         if (menu) {
2517             menu.style.left = `${e.originalEvent.clientX}px`;
2518             menu.style.top = `${e.originalEvent.clientY}px`;
2519             menu.style.display = 'block';
2520
2521             // Mostra/esconde opções conforme o contexto
2522             menu.querySelectorAll('button').forEach(btn => {
2523                 const acao = btn.dataset.action;
2524
2525                 if (['rename-node', 'delete-node', 'start-edge'].includes(acao)) {
2526                     btn.style.display = this.estado.ctxVerticeId ? 'block' : 'none';
2527                 } else if (['edit-edge', 'delete-edge'].includes(acao)) {
2528                     btn.style.display = this.estado.ctxArestaId ? 'block' : 'none';
2529                 } else if (acao === 'add-node') {
2530                     btn.style.display = !this.estado.ctxVerticeId && !this.estado.ctxArestaId ?
2531                         'block' : 'none';
2532                 } else if (acao === 'cancel-edge') {
2533                     btn.style.display = this.estado.arestaOrigem ? 'block' : 'none';
2534                 }
2535             });
2536         }
2537     });
2538
2539     /** Conecta eventos dos botões e controles da interface */
2540     conectarEventosInterface() {
2541         // Botões de modo
2542         pegarElemento('modeAddVertex')?.addEventListener('click', () => this.definirModo(
2543             MODOS.VERTECE));
2544         pegarElemento('modeAddEdge')?.addEventListener('click', () => this.definirModo(
2545             MODOS.ARESTA));
2546         pegarElemento('modeSelect')?.addEventListener('click', () => this.definirModo(
2547             MODOS.SELECIONAR));
2548
2549         // Configurações do grafo
2550         pegarElemento('selTipo')?.addEventListener('change', (e) => {
2551             this.estado.orientado = e.target.value === TIPOS_GRAFO.ORIENTADO;
2552             this.atualizarEstiloArestas();
2553             registrarLog(`Tipo: ${this.estado.orientado ? 'Orientado' : 'Não orientado'}`);
2554         });
2555
2556         pegarElemento('selPeso')?.addEventListener('change', (e) => {
2557             this.estado.ponderado = e.target.value === 'sim';
2558             this.atualizarEstiloArestas();
2559             registrarLog(`Peso: ${this.estado.ponderado ? 'sim (duplo clique para editar)' :
2560                 'não'}`);
2561         });
2562
2563         pegarElemento('swLoopAllowed')?.addEventListener('change', (e) => {

```

```

2560     this.estado.permitirLacos = e.target.checked;
2561     registrarLog(`Loops: ${this.estado.permitirLacos ? 'sim' : 'não'}`);
2562 });
2563
2564 pegarElemento('swCurvedEdges')?.addEventListener('change', (e) => {
2565     this.estado.arestasCurvas = e.target.checked;
2566     this.atualizarEstiloArestas();
2567 });
2568
2569 // Operações de vértice
2570 pegarElemento('btnAddV')?.addEventListener('click', () => {
2571     this.adicionarVertice(pegarElemento('inpV').value);
2572 });
2573
2574 pegarElemento('btnRemV')?.addEventListener('click', () => {
2575     this.removerVertice(pegarElemento('inpV').value);
2576 });
2577
2578 // Operações de aresta
2579 pegarElemento('btnAddE')?.addEventListener('click', () => {
2580     this.adicionarAresta(
2581         pegarElemento('inpU').value,
2582         pegarElemento('inpW').value,
2583         pegarElemento('inpPeso').value
2584     );
2585 });
2586
2587 pegarElemento('btnRemE')?.addEventListener('click', () => {
2588     const origem = pegarElemento('inpU').value;
2589     const destino = pegarElemento('inpW').value;
2590     const arestas = this.cy.edges().filter(e =>
2591         (e.data('source') === origem && e.data('target') === destino) ||
2592         (!this.estado.orientado && e.data('source') === destino && e.data('target') ===
2593             origem)
2594     );
2595     if (arestas.length) {
2596         arestas.remove();
2597         this.atualizarIndicadores();
2598         registrarLog(`🗑️ Aresta ${origem}→${destino} removida.`);
2599     }
2600 });
2601
2602 // Criação em lote
2603 pegarElemento('btnBatchAdd')?.addEventListener('click', () => {
2604     const quantidade = Number(pegarElemento('inpBatchN').value);
2605     const tipo = pegarElemento('selBatchTipo').value;
2606     const prefixo = pegarElemento('inpPrefixo').value;
2607
2608     for (let i = 0; i < quantidade; i++) {
2609         let id;
2610         if (tipo === 'alfabetico') {
2611             id = String.fromCharCode(65 + (i % 26));
2612         } else if (tipo === 'numerico') {
2613             id = String(i);
2614         } else {
2615             id = `${prefixo}${i}`;
2616         }
2617
2618         if (this.cy.getElementById(id).length === 0) {
2619             this.adicionarVertice(id);
2620         }
2621     }
2622 });
2623
2624 // Arestas automáticas
2625 pegarElemento('btnAutoEdges')?.addEventListener('click', () => {
2626     const vertices = this.obterVertices();
2627     if (vertices.length < 2) {
2628         exibirNotificacao('Crie pelo menos 2 vértices', NOTIFICACAO.AVISO);
2629     }
2630 });

```



```

2628         return;
2629     }
2630
2631     const modo = pegarElemento('selAutoEdgesMode').value;
2632
2633     if (modo === 'caminho') {
2634         for (let i = 0; i < vertices.length - 1; i++) {
2635             this.adicionarAresta(vertices[i], vertices[i + 1], null, false);
2636         }
2637     } else if (modo === 'ciclo') {
2638         for (let i = 0; i < vertices.length - 1; i++) {
2639             this.adicionarAresta(vertices[i], vertices[i + 1], null, false);
2640         }
2641         this.adicionarAresta(vertices[vertices.length - 1], vertices[0], null, false);
2642     } else if (modo === 'completo') {
2643         for (let i = 0; i < vertices.length; i++) {
2644             for (let j = i + 1; j < vertices.length; j++) {
2645                 this.adicionarAresta(vertices[i], vertices[j], null, false);
2646             }
2647         }
2648     } else if (modo === 'aleatorio') {
2649         const quantidade = Number(pegarElemento('inpAutoEdgesM').value);
2650         for (let k = 0; k < quantidade; k++) {
2651             const i = Math.floor(Math.random() * vertices.length);
2652             const j = Math.floor(Math.random() * vertices.length);
2653             if (i !== j) {
2654                 this.adicionarAresta(vertices[i], vertices[j], null, false);
2655             }
2656         }
2657     }
2658
2659     this.atualizarIndicadores();
2660     exibirNotificacao('Arestas geradas com sucesso', NOTIFICACAO.SUCESSO);
2661 });
2662
2663 pegarElemento('btnAutoEdgesClear')?.addEventListener('click', () => {
2664     this.cy.edges().remove();
2665     this.atualizarIndicadores();
2666 });
2667
2668 // Botões de coloração
2669 pegarElemento('btnColorirVertices')?.addEventListener('click', () => {
2670     if (!this.estado.executandoAnimacao && this.cy.nodes().length) {
2671         const resultado = this.colorirVerticesWelshPowell();
2672         this.aplicarCoresVertices(resultado.cores);
2673     }
2674 });
2675
2676 pegarElemento('btnColorirArestas')?.addEventListener('click', () => {
2677     if (!this.estado.executandoAnimacao && this.cy.edges().length) {
2678         const resultado = this.colorirArestasOtimizado();
2679         this.aplicarCoresArestas(resultado.cores);
2680     }
2681 });
2682
2683 pegarElemento('btnColorirTudo')?.addEventListener('click', () => {
2684     if (!this.estado.executandoAnimacao) {
2685         this.colorirGrafoCompleto();
2686     }
2687 });
2688
2689 pegarElemento('btnColorirAnimado')?.addEventListener('click', () => {
2690     if (!this.estado.executandoAnimacao) {
2691         this.executarColoracaoAnimada();
2692     }
2693 });
2694
2695 pegarElemento('btnColorirReset')?.addEventListener('click', () => {
2696     if (!this.estado.executandoAnimacao) {

```



```

2697         this.resetarCores();
2698     }
2699 });
2700
2701 // Modelos
2702 pegarElemento('btnCarregarModelo')?.addEventListener('click', () => {
2703     this.carregarModelo(pegarElemento('selModelo').value);
2704 });
2705
2706 // Ações gerais
2707 pegarElemento('btnClear')?.addEventListener('click', () => this.limparTudo());
2708 pegarElemento('btnStabilize')?.addEventListener('click', () => this.cy.fit(undefined
, 50));
2709
2710 pegarElemento('btnResetColors')?.addEventListener('click', () =>
this.limparCoresBusca());
2711
2712 // Undo/Redo
2713 pegarElemento('btnUndo')?.addEventListener('click', () => {
2714     const acao = this.historico.desfazer();
2715     if (acao && acao.desfazer) {
2716         acao.desfazer();
2717         registrarLog('↶ Desfazer');
2718     }
2719 });
2720
2721 pegarElemento('btnRedo')?.addEventListener('click', () => {
2722     const acao = this.historico.refazer();
2723     if (acao && acao.refazer) {
2724         acao.refazer();
2725         registrarLog('↷ Refazer');
2726     }
2727 });
2728
2729 // Arquivo
2730 pegarElemento('btnSaveFile')?.addEventListener('click', () => {
2731     const conteudo = this.exportarParaTexto();
2732     const blob = new Blob([conteudo], { type: 'text/plain' });
2733     const link = document.createElement('a');
2734     link.href = URL.createObjectURL(blob);
2735     link.download = pegarElemento('inpFileName').value || 'grafo.txt';
2736     link.click();
2737     URL.revokeObjectURL(link.href);
2738     registrarLog('📁 Grafo salvo como arquivo .txt');
2739     exibirNotificacao('Arquivo salvo com sucesso', NOTIFICACAO.SUCESSO);
2740 });
2741
2742 pegarElemento('btnLoadFile')?.addEventListener('click', () => {
2743     pegarElemento('fileInput').click();
2744 });
2745
2746 pegarElemento('fileInput')?.addEventListener('change', (e) => {
2747     const arquivo = e.target.files[0];
2748     if (!arquivo) return;
2749
2750     const leitor = new FileReader();
2751     leitor.onload = (ev) => this.importarDeTexto(ev.target.result);
2752     leitor.readAsText(arquivo);
2753     e.target.value = '';
2754 });
2755
2756 // Velocidade
2757 pegarElemento('btnSpeedDown')?.addEventListener('click', () => {
2758     this.estado.velocidadeAnimacao = Math.max(100, this.estado.velocidadeAnimacao - 50
);
2759     pegarElemento('speedValue').textContent = this.estado.velocidadeAnimacao;
2760 });
2761
2762 pegarElemento('btnSpeedUp')?.addEventListener('click', () => {
2763     this.estado.velocidadeAnimacao = Math.min(2000, this.estado.velocidadeAnimacao +

```

```

50);
2763 pegarElemento('speedValue').textContent = this.estado.velocidadeAnimacao;
2764 });
2765
2766 pegarElemento('btnSpeedReset')?.addEventListener('click', () => {
2767     this.estado.velocidadeAnimacao = 1050;
2768     pegarElemento('speedValue').textContent = 1050;
2769 });
2770
2771 // Painel de Logs
2772 pegarElemento('btnClearLog')?.addEventListener('click', () => {
2773     if (painelLogConteudo) painelLogConteudo.textContent = '';
2774     if (caixaLog && caixaLog !== document.createElement('div')) caixaLog.textContent =
        '';
2775     registrarLog('🧹 Logs limpos.');
```

2776 });

2777

2778 pegarElemento('btnToggleLog')?.addEventListener('click', () => {

2779 const conteudo = pegarElemento('logPanelContent');

2780 const icone = pegarElemento('btnToggleLog').querySelector('i');

2781 const painel = pegarElemento('logPanel');

2782

2783 if (conteudo.style.display === 'none') {

2784 conteudo.style.display = 'block';

2785 icone.className = 'bi bi-chevron-up';

2786 painel.style.maxHeight = '400px';

2787 painel.style.minHeight = '300px';

2788 } else {

2789 conteudo.style.display = 'none';

2790 icone.className = 'bi bi-chevron-down';

2791 painel.style.maxHeight = '50px';

2792 painel.style.minHeight = '50px';

2793 }

2794 });

2795

2796 // Menu de contexto

2797 pegarElemento('ctxMenu')?.addEventListener('click', (e) => {

2798 const botao = e.target.closest('button');

2799 if (!botao) return;

2800

2801 const acao = botao.dataset.action;

2802

2803 const acoes = {

2804 'add-node': () => {

2805 if (this.estado.ctxPosicaoTela) {

2806 this.adicionarVertice(this.proximoIdVertice(), this.estado.ctxPosicaoTela);

2807 }

2808 },

2809 'rename-node': () => {

2810 if (this.estado.ctxVerticeId) {

2811 const novo = prompt('Novo nome do vértice:', this.estado.ctxVerticeId);

2812 if (novo && novo !== this.estado.ctxVerticeId) {

2813 this.renamearVertice(this.estado.ctxVerticeId, novo);

2814 }

2815 }

2816 },

2817 'delete-node': () => {

2818 if (this.estado.ctxVerticeId) {

2819 this.removerVertice(this.estado.ctxVerticeId);

2820 }

2821 },

2822 'start-edge': () => {

2823 if (this.estado.ctxVerticeId) {

2824 this.estado.arestaOrigem = this.estado.ctxVerticeId;

2825 this.definirModo(MODOS.ARESTA);

2826 registrarLog(`🔗 Iniciando aresta de \${this.estado.ctxVerticeId}`);

2827 }

2828 },

2829 'delete-edge': () => {

```

2830         if (this.estado.ctxArestaId) {
2831             this.removerArestaPorId(this.estado.ctxArestaId);
2832         }
2833     },
2834     'edit-edge': () => {
2835         if (this.estado.ctxArestaId && this.estado.ponderado) {
2836             this.editarPesoAresta(this.estado.ctxArestaId);
2837         }
2838     },
2839     'cancel-edge': () => {
2840         this.estado.arestaOrigem = null;
2841         registrarLog('Criação cancelada.');
```

```

2899     if (ev.target.tagName === 'INPUT' || ev.target.tagName === 'TEXTAREA' ||
2900         ev.target.tagName === 'SELECT') {
2901         return;
2902     }
2903
2904     // Ctrl+Z: Desfazer
2905     if (ev.ctrlKey && ev.key === 'z' && !ev.shiftKey && !
2906         this.estado.executandoAnimacao) {
2907         ev.preventDefault();
2908         const acao = this.historico.desfazer();
2909         if (acao && acao.desfazer) {
2910             acao.desfazer();
2911             registrarLog('↶ Desfazer (Ctrl+Z)');
2912         }
2913     }
2914
2915     // Ctrl+Y: Refazer
2916     if ((ev.ctrlKey && ev.key === 'y') || (ev.ctrlKey && ev.shiftKey && ev.key === 'z'
2917     )) {
2918         if (!this.estado.executandoAnimacao) {
2919             ev.preventDefault();
2920             const acao = this.historico.refazer();
2921             if (acao && acao.refazer) {
2922                 acao.refazer();
2923                 registrarLog('↷ Refazer (Ctrl+Y)');
2924             }
2925         }
2926     }
2927
2928     // Delete: Remover selecionado
2929     if (ev.key === 'Delete' && !this.estado.executandoAnimacao) {
2930         if (this.estado.verticeSelecionadoId) {
2931             this.removerVertice(this.estado.verticeSelecionadoId);
2932         } else if (this.estado.arestaSelecionadaId) {
2933             this.removerArestaPorId(this.estado.arestaSelecionadaId);
2934         }
2935     }
2936
2937     // Escape: Cancelar criação de aresta
2938     if (ev.key === 'Escape') {
2939         if (this.estado.arestaOrigem) {
2940             this.estado.arestaOrigem = null;
2941             registrarLog('Criação de aresta cancelada.');
```